Plan and implement data platform resources

At a glance

- **Level**

IntermediateIntermediate

- **Product**

AzureAzure SQL DatabaseSQL ServerAzureAzure SQL DatabaseSQL Server

- **Role**

Data EngineerDatabase AdministratorData EngineerDatabase Administrator

- **Subject**

Data engineeringData managementData engineeringData management

- Deploy data platform resources in Azure SQL.
- Explore options for deployment and migration.
- Calculate resource requirements and create templates.

Prerequisites

- Familiarity with on-premises virtualization technologies, including: VMs, virtual networking, and virtual hard disks.
- Familiarity with network configuration, including TCP/IP, Domain Name System (DNS), virtual private networks (VPNs), firewalls, and encryption technologies.
- Ability to create an on-premises SQL Server database, including the configuration of data files and log files.
- Ability to write code in the SQL language, particularly the Microsoft T-SQL dialect, at a basic level.
- Experience creating and configuring resources using the Azure portal.

Modules in this learning path

Deploy IaaS solutions with Azure SQLDeploy IaaS solutions with Azure SQL

- Configure virtual machine sizing, storage, and networking options to ensure adequate performance for your database workloads.
- Choose and configure appropriate high availability options.
- Explain IaaS options to deploy SQL Server in Azure
- Understand hybrid scenarios
- Explore performance and security
- Explain high availability and disaster recovery options

• Exercise: Provision a SQL Server on an Azure Virtual Machine

Feature	Ultra Disk	Premium SSD v2	Premium SSD	Standard SSD	Standard HDD
Disk type	SSD	SSD	SSD	SSD	HDD
Best for	IO-intensive workloads	Performance-sensitive workloads	Performance-sensitive workloads	Lightweight workloads	Backups, non-critical workloads
Max disk size	65,536 GiB	65,536 GiB	32,767 GiB	32,767 GiB	32,767 GiB
Max throughput	10,000 MB/s	2,000 MB/s	900 MB/s	750 MB/s	500 MB/s
Max IOPS	400,000	80,000	20,000	6,000	2,000

Summary

- **1 minute**
- Azure provides a great deal of flexibility for deploying SQL Server to an Azure virtual machine.
- The hybrid licensing options reduce your cost.
- It also allows for additional protection for high availability and disaster recovery as part of your software assurance benefits.
- Azure Resource Manager offers many ways to deploy your resources in both a programmatic and graphical fashion.
- Now that you've reviewed this module, you should be able to:
- Explore the basics of SQL Server in an Infrastructure as a Service (IaaS) offering
- **Learn how hybrid scenario works**
- Explore performance and security options available
- Understand the high availability and disaster recovery options

Deploy IaaS solutions with Azure SQL Deploy IaaS solutions with Azure SQL

Introduction

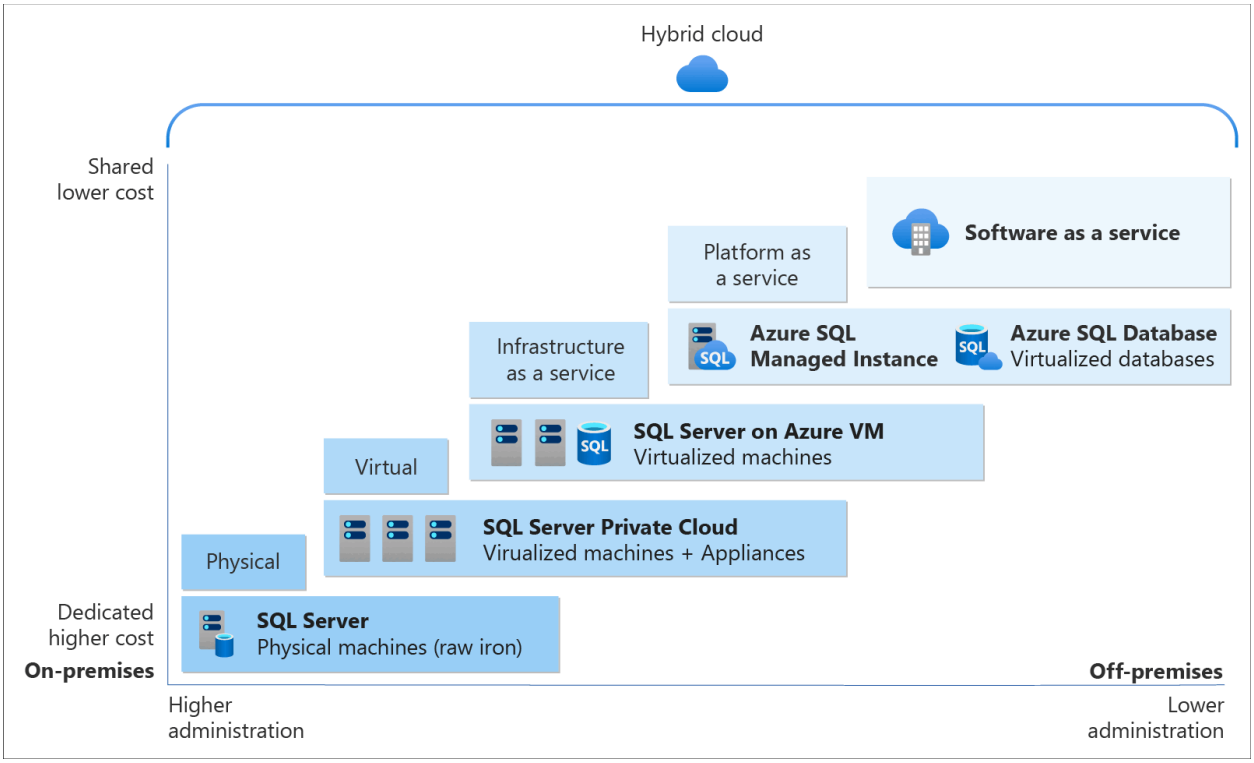
- **1 minute**
- The Platform as a Service (PaaS) offering for SQL Server can be a great solution for certain workloads.
- The PaaS offering provides less granular control over the infrastructure.
- It also relegates management of the underlying components (memory, CPU, storage, operating system, etc.) to Microsoft Azure.
- This module focuses on ways to provision and deploy Azure SQL Database and Azure SQL managed instances, as well as provide guidance on the various options when performing a migration to these platforms.

Learning objectives

- At the end of this module, you'll be able to:
- Understand PaaS provisioning and deployment options
- Understand elastic pools and hyperscale features

• Examine SQL Managed Instances

- Explain PaaS options for deploying SQL Server in Azure
- Platform as a Service (PaaS) provides a complete development and deployment environment in the cloud, which can be used for simple cloud-based applications and for advanced enterprise applications.
- Azure SQL Database and Azure SQL Managed Instance are part of PaaS offering for Azure SQL.
- Azure SQL Database – Part of a family of products built upon the SQL Server engine, in the cloud.
- It gives developers a great deal of flexibility in building new application services, and granular deployment options at scale.
- SQL Database offers a low maintenance solution that can be a great option for certain workloads.
- Azure SQL Managed Instance– It's best for most migration scenarios to the cloud as it provides fully managed services and capabilities.



- Each offering provides a certain level of administration you have over the infrastructure, by the degree of cost efficiency.

Deployment models

- Azure SQL Database is available in two different deployment models:
- Single database – A single database that is billed and managed on a per-database level.
- You manage each of your databases individually from scale and data size perspectives.
- Each database deployed in this model has its own dedicated resources, even if deployed to the same logical server.
- Elastic Pools – A group of databases that are managed together and share a common set of resources.
- Elastic pools provide a cost-effective solution for software as a service application model, as resources are shared between all databases.
- You can configure resources based either on the DTU-based purchasing model or the vCore-based purchasing model.

Purchasing model

- In Azure, all services support physical hardware, and you have the flexibility to choose between two different purchasing models:

Database Transaction Unit (DTU)

- DTU-based purchasing model is calculated based on a formula combining compute, storage, and I/O resources.
- It's a good choice for customers who want simple, preconfigured resource options.
- The DTU purchasing model comes in three different service tiers: Basic, Standard, and Premium.
- Each tier has varying capabilities, providing a wide range of options when choosing this platform.
- In terms of performance, the Basic tier is used for less demanding workloads, while the Premium tier is used for intensive workload requirements.
- Compute and storage resources are dependent on the DTU level, and they provide a range of performance capabilities at a fixed storage limit, backup retention, and cost.
- For more information about DTU purchasing model, see DTU-based purchasing model overview.

vCore

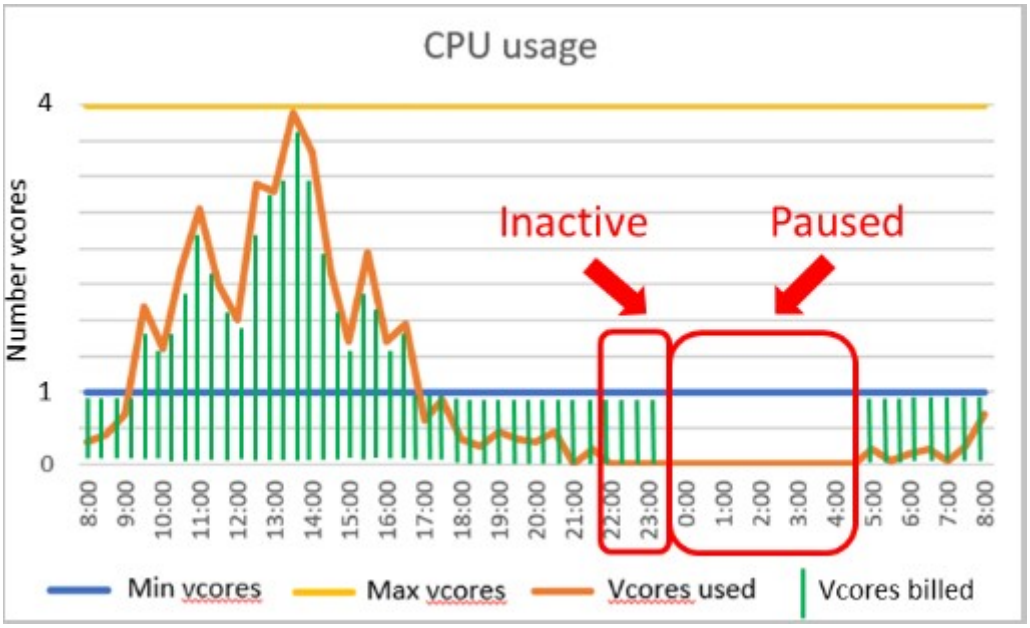
- The vCore-based purchasing model allows you to purchase a specified number of vCores based on your given workloads. vCore is the default purchasing model when purchasing Azure SQL Database resources. vCore databases have a specific relationship between the number of cores and the amount of memory and storage provided to the database.
- The vCore purchasing model is supported by both Azure SQL Database and Azure SQL Managed Instance.
- You can purchase vCore databases in three different service tiers as well:

Service Tier	Best For	Storage Type	Latency	Compute Tiers	Resilience Features	Max Database Size
General Purpose	General-purpose workloads	Azure premium storage	Higher than BC	Provisioned, Serverless	N/A	4 TB
Business Critical	High-performing workloads	Local SSDs	Lowest	Provisioned	Built-in read-only replica, highest resilience to failure	4 TB
Hyperscale	Large-scale databases	Azure premium storage	Varies	Provisioned	Unique architecture for scaling	100 TB

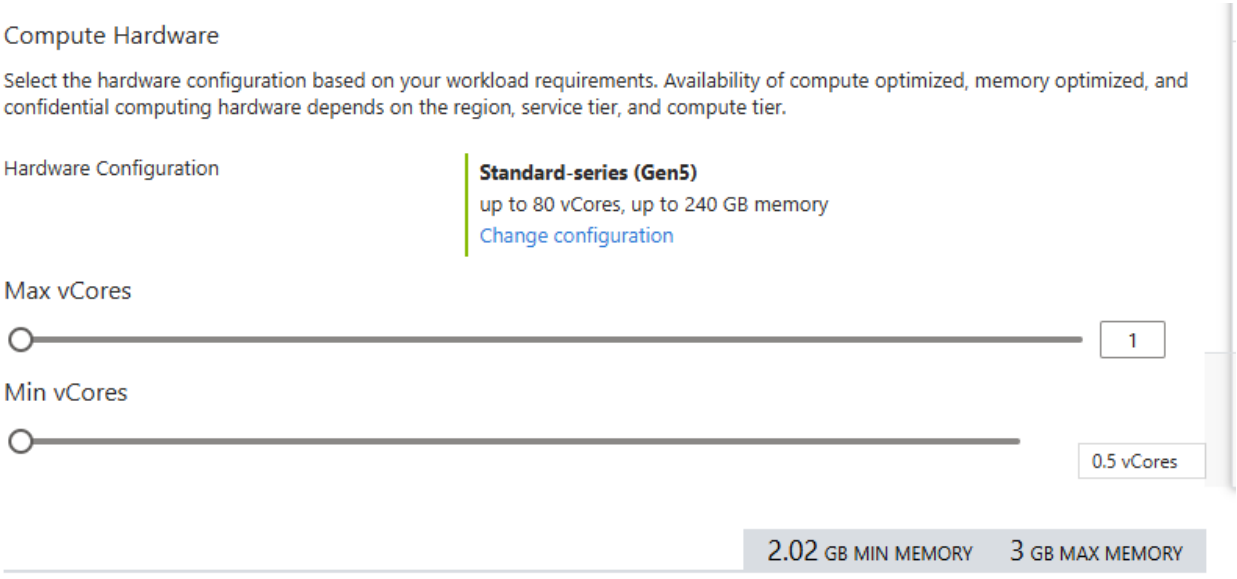
- The General Purpose service tier offers two compute options: Provisioned and Serverless.
- Provisioned resources are preallocated and billed hourly based on vCores configured, ideal for consistent workloads.
- Serverless resources autoscale based on demand and are billed per second, making it cost-efficient for variable workloads.

Serverless

- The term "Serverless" can be misleading because you still deploy your Azure SQL Database to a logical server for connection.
- Serverless is a compute tier that automatically scales resources up or down based on workload demand.
- When the workload no longer requires compute resources, the database is paused, and only storage is billed during the inactive period.
- Upon a connection attempt, the database resumes and becomes available.



- The setting to control pausing is called the autopause delay, with a minimum value of 60 minutes and a maximum value of seven days.
- If the database remains idle for that duration, it pauses.
- Once the database is inactive for the specified time, it pauses until a subsequent connection attempt.
- Configuring a compute autoscaling range and an autopause delay affects database performance and compute costs.
- Applications using serverless should be configured to handle connection errors and include retry logic, as connecting to a paused database generates a connection error.
- Another difference between serverless and the standard vCore model of Azure SQL Database is that with serverless, you can specify a minimum and maximum number of vCores.
- Memory and I/O limits are proportional to the specified range.



- The image shows the configuration screen for a serverless database in the Azure portal.
- You have the option to select a minimum as low as half of a vCore and a maximum as high as 16 vCores.
- Serverless isn't fully compatible with all the features in Azure SQL Database since some of them require background processes to run constantly, such as:

- **Geo-replication**
- **Long-term backup retention**
- **A job database in elastic jobs**
- The sync database in SQL Data Sync (Data Sync is a service that replicates data between a group of databases)

Note

- The serverless compute tier is available in both the General Purpose and Hyperscale service tiers of the vCore purchasing model.
- Auto-pause and auto-resume are currently only supported in the General Purpose tier.

Backups

- One of the most important features of the Platform as a Service offering is backups.
- In this case, the system automatically performs backups without any intervention from you.
- Azure blob geo-redundant storage stores these backups, and by default, retains them for between 7 and 35 days, depending on the service tier of the database.
- Basic and vCore databases default to seven days of retention, and administrators can adjust this value for vCore databases.
- You can extend the retention time by configuring long-term retention (LTR), allowing you to retain backups for up to 10 years.
- In order to provide redundancy, you're also able to use read-accessible geo-redundant blob storage.
- This storage would replicate your database backups to a secondary region of your preference.
- It would also allow you to read from that secondary region if needed.
- Manual backups of databases aren't supported, and the platform will deny any request to do so.
- Database Backups are taken on a given schedule:
- **Full – Once a week**
- **Differential – Every 12 hours**
- Log – Every 5-10 minutes depending on transaction log activity
- This backup schedule should meet the needs of most recovery point/time objectives (RPO/RTO), however, each customer should evaluate whether they meet your business requirements.
- There are several options available for restoring a database.
- Due to the nature of Platform as a Service, you can't manually restore a database using conventional methods, such as issuing the T-SQL command RESTORE DATABASE.
- Regardless of which restore method is implemented, it isn't possible to restore over an existing database.
- If a database needs to be restored, the existing database must be dropped or renamed prior to initiating the restore process.
- Furthermore, keep in mind that depending on the platform service tier, restore times aren't guaranteed and could fluctuate.
- It's recommended that you test the restore process to obtain a baseline metric on how long a restore could potentially take.
- Restore using the Azure portal – Using the Azure portal you have the option of restoring a database to the same logical server for Azure SQL Database, or you can use the restore to create a new database on a new server in any Azure region.
- Restore using scripting Languages – Both PowerShell and Azure CLI can be utilized in order to restore a database.

Note

- Copy-only backup to Azure blob storage is available for SQL Managed Instance.
- SQL Database doesn't support this feature.

- For more information about automated backups, see Automated backups - Azure SQL Database & Azure SQL Managed Instance.

Active geo-replication

- Geo-replication is a business continuity feature that asynchronously replicates a database to up to four secondary replicas.
- As transactions are committed to the primary (and its replicas within the same region), the transactions are sent to the secondaries to be replayed.
- Since this communication is done asynchronously, the calling application doesn't have to wait for the secondary replica to commit the transaction before SQL Server returns control to the caller.
- The secondary databases are readable and can be used to offload read-only workloads, thus freeing up resources for transactional workloads on the primary or placing data closer to your end users.
- Furthermore, the secondary databases can be in the same region as the primary or in another Azure region
- You can initiate a failover either manually by the user or programmatically by the application.
- If a failover occurs, you can update the application connection strings to reflect the new endpoint of what is now the primary database.

Failover groups

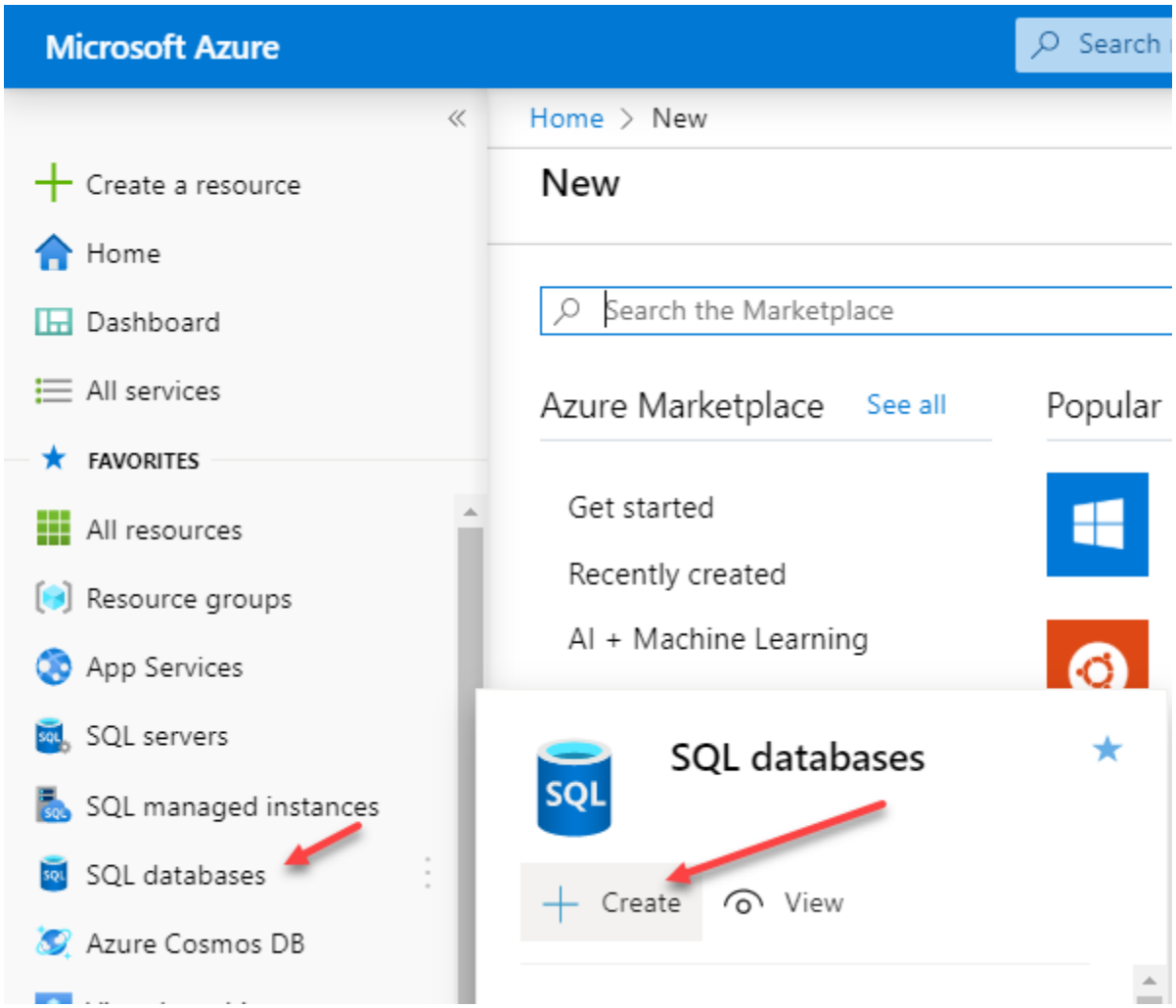
- Failover groups are built on top of the technology used in geo-replication but provide a single endpoint for connection.
- The major reason for using failover groups is that they provide endpoints that can be utilized to route traffic to the appropriate replica.
- Your application can then connect after a failover without connection string changes.

Explore single SQL database

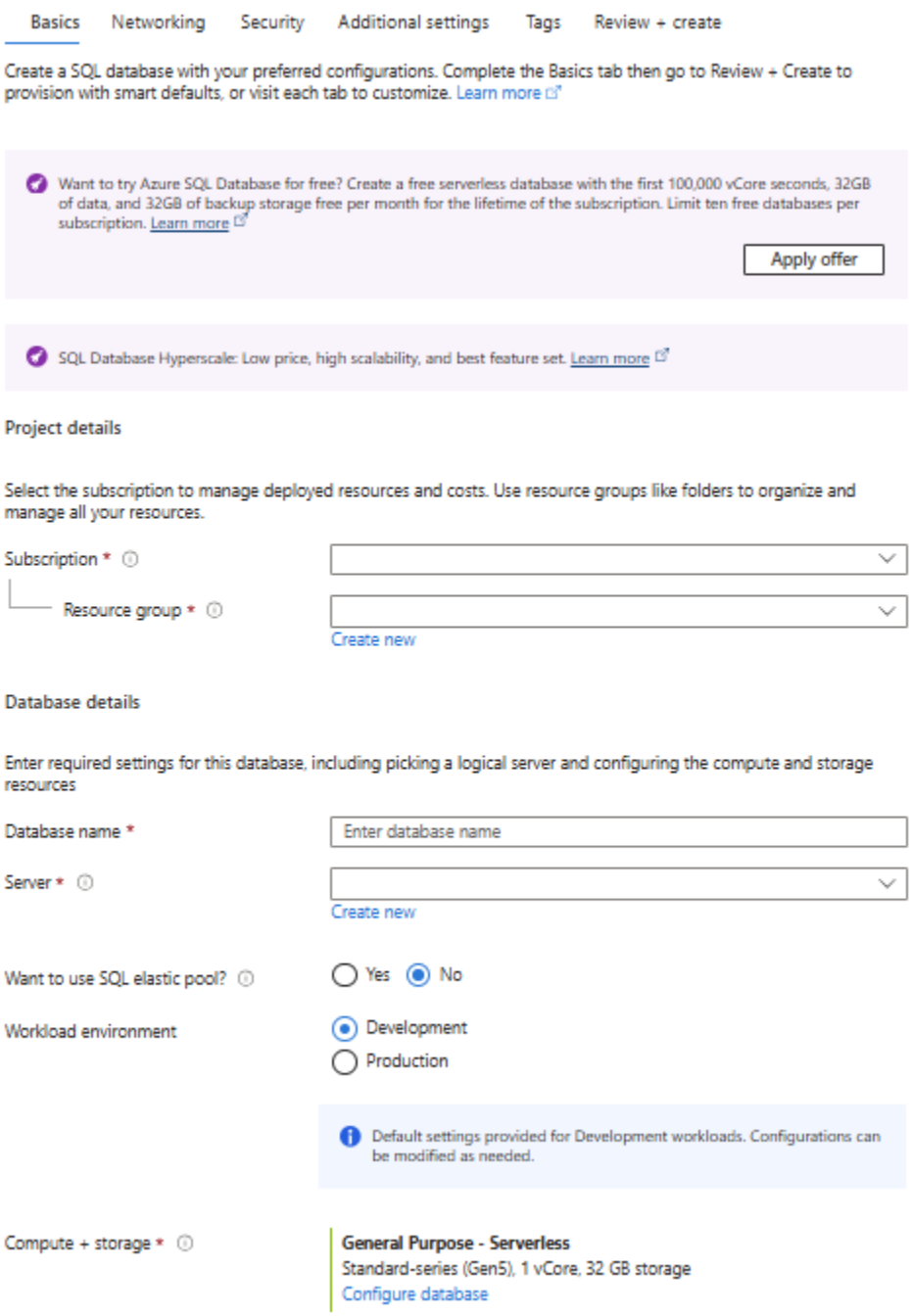
- Let's look at several methods for deploying a single Azure SQL Database.

Deploying via the portal

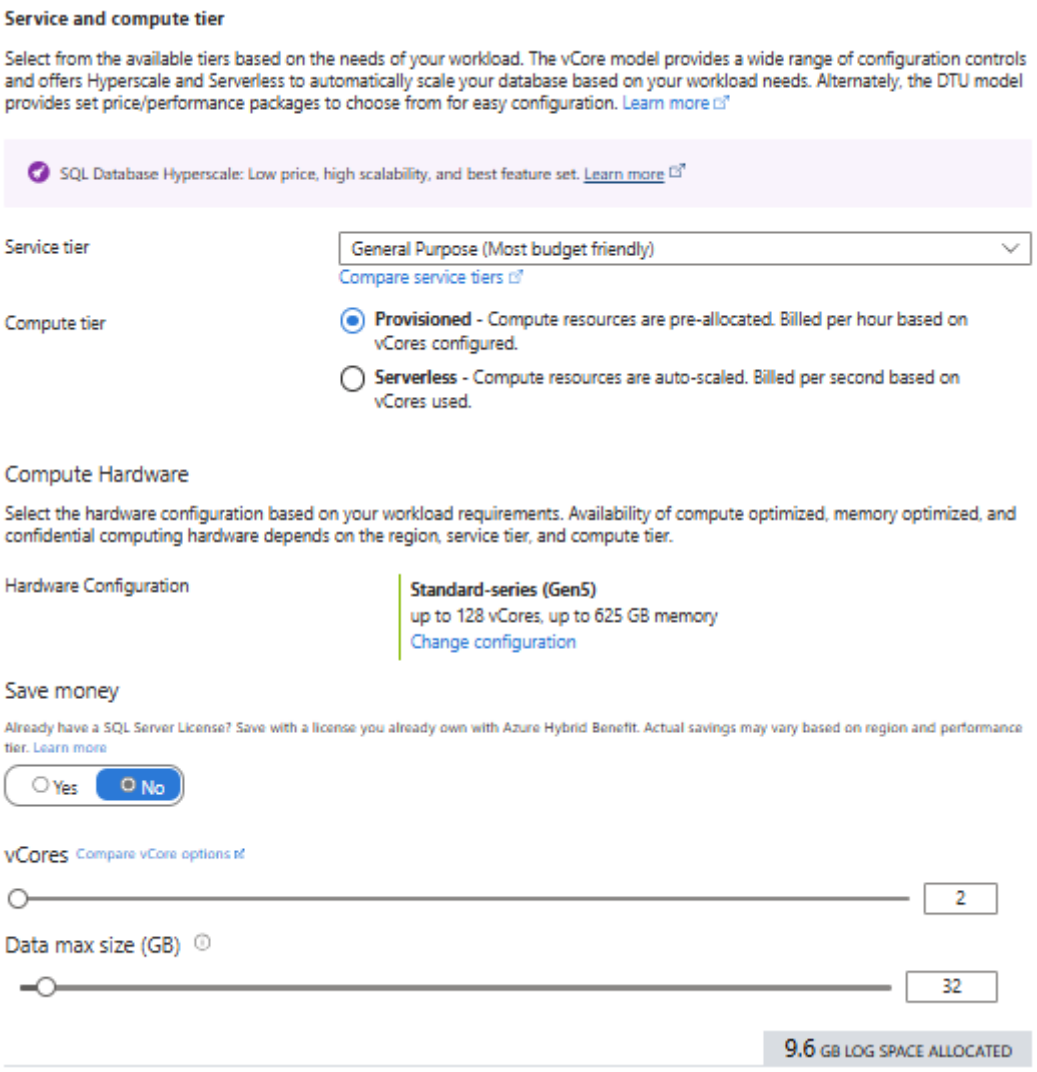
- Creating an SQL database through the Azure portal is simple.
- First, navigate to the Azure portal and select "SQL Databases" from the left-hand menu.
- In the slide-out panel that appears, select "Create."



- In the pane in the image below, you'll notice that the subscription should already be provided for you.
- You'll need to supply the following information:
- Resource Group – If you have an existing resource group you wish to use, select it from the drop-down list.
- To create a new resource group for this Azure SQL Database, choose the Create new option.
- Database Name – Provide a name for your database.
- Server – Each database must reside on a logical server.
- If you already have one in the appropriate region, you can select it.
- Otherwise, select the Create new link and follow the prompts to create a new logical server to host the database.
- Want to use SQL elastic pool? – Decide whether to use an elastic pool.
- Compute + storage – Determine the appropriate compute resources needed.
- By default, it's set to Gen5, 2vCore, with 32 GB of storage.
- Select Configure database to view and choose alternate configuration options.



- Here, you'll notice that the service tier is General Purpose and the compute tier is Serverless, as discussed previously.
- Serverless is billed per second based on the number of vCores in use.
- The alternative option is Provisioned, where compute resources are preallocated and billed per hour based on the number of configured vCores.



- Deploying an Azure SQL Database via PowerShell/CLI
- You can also deploy your database using Azure PowerShell or the Azure CLI.
- The image below shows the PowerShell example where you're creating a new resource group, and defining an admin called SqlAdmin and then creating a new server, database, and firewall rule.

PowerShell

```
# Connect-AzAccount

# The SubscriptionId in which to create these objects
$SubscriptionId = ''

# Set the resource group name and location for your server
$resourceGroupName = "myResourceGroup-$(Get-Random)"
$location = "westus2"

# Set an admin login and password for your server
$adminSqlLogin = "SqlAdmin"
$password = "ChangeYourAdminPassword1"

# Set server name - the logical server name has to be unique in the system
$serverName = "server-$(Get-Random)"
```

```
# The sample database name
$databaseName = "mySampleDatabase"

# The ip address range that you want to allow to access your server
$startIp = "0.0.0.0"
$endIp = "0.0.0.0"

# Set subscription
Set-AzContext -SubscriptionId $subscriptionId

# Create a resource group
$resourceGroup = New-AzResourceGroup -Name $resourceGroupName -Location $location

# Create a server with a system wide unique server name
$server = New-AzSqlServer -ResourceGroupName $resourceGroupName `
  -ServerName $serverName `
  -Location $location `
• -SqlAdministratorCredentials $(New-Object -TypeName System.Management.Automation.PSCredential
   -ArgumentList $adminSqlLogin, $(ConvertTo-SecureString -String $password -AsPlainText -Force))

# Create a server firewall rule that allows access from the specified IP range

$serverFirewallRule = New-AzSqlServerFirewallRule -ResourceGroupName $resourceGroupName `
  -ServerName $serverName `
• -FirewallRuleName "AllowedIPs" -StartIpAddress $startIp -EndIpAddress $endIp

# Create a blank database with an S0 performance level

$database = New-AzSqlDatabase -ResourceGroupName $resourceGroupName `
  -ServerName $serverName `
  -DatabaseName $databaseName `
  -RequestedServiceObjectiveName "S0" `
-SampleName "AdventureWorksLT"
```

• The Azure CLI can also be used to deploy an Azure SQL Database as shown below:

```
  Bash
#!/bin/bash

# set execution context (if necessary)
az account set --subscription <replace with your subscription name or id>

# Set the resource group name and location for your server
  resourceGroupName=myResourceGroup-$RANDOM
  location=westus2

# Set an admin login and password for your database
  adminlogin=ServerAdmin
  password=`openssl rand -base64 16`
```

```
# password=<EnterYourComplexPasswordHere1>

# The logical server name has to be unique in all of Azure
servername=server-$RANDOM

# The ip address range that you want to allow to access your DB
startip=0.0.0.0
endip=0.0.0.0

# Create a resource group
az group create \
 --name $resourceGroupName \
 --location $location

# Create a logical server in the resource group
az sql server create \
 --name $servername \
 --resource-group $resourceGroupName \
 --location $location \
 --admin-user $adminlogin \
 --admin-password $password

# Configure a firewall rule for the server

az sql server firewall-rule create \
 --resource-group $resourceGroupName \
 --server $servername \
 -n AllowYourIp \
 --start-ip-address $startip \
 --end-ip-address $endip

# Create a database in the server
az sql db create \
 --resource-group $resourceGroupName \
 --server $servername \
 --name mySampleDatabase \
 --sample-name AdventureWorksLT \
 --edition GeneralPurpose \
 --family Gen5 \
 --capacity 1

# Echo random password
```

echo $password

- Deploying Azure SQL Database Using Azure Resource Manager templates
- Another method for deploying resources is as mentioned earlier using an Azure Resource Manager template.
- A Resource Manager template gives you the most granular control over your resources, and Microsoft provides a GitHub repository called Azure-Quickstart-Templates, which hosts Azure Resource Manager templates that you can reference in your deployments.
- A PowerShell example of deploying a GitHub based template is shown below:

```
PowerShell

#Define Variables for parameters to pass to template
$projectName = Read-Host -Prompt "Enter a project name"
$location = Read-Host -Prompt "Enter an Azure location (i.e. centralus)"
```

```
$adminUser = Read-Host -Prompt "Enter the SQL server administrator username"
$adminPassword = Read-Host -Prompt "Enter the SQl server administrator password" -AsSecureString
$resourceGroupName = "${projectName}rg"

#Create Resource Group and Deploy Template to Resource Group

New-AzResourceGroup -Name $resourceGroupName -Location $location

New-AzResourceGroupDeployment -ResourceGroupName $resourceGroupName `
-TemplateUri "https://raw.githubusercontent.com/Azure/azure-quickstart-templates/master/quickstarts/microsoft.sql/sql-logical-server/azuredeploy.json" `
-administratorLogin $adminUser -administratorLoginPassword $adminPassword


Read-Host -Prompt "Press [ENTER] to continue ..."
```

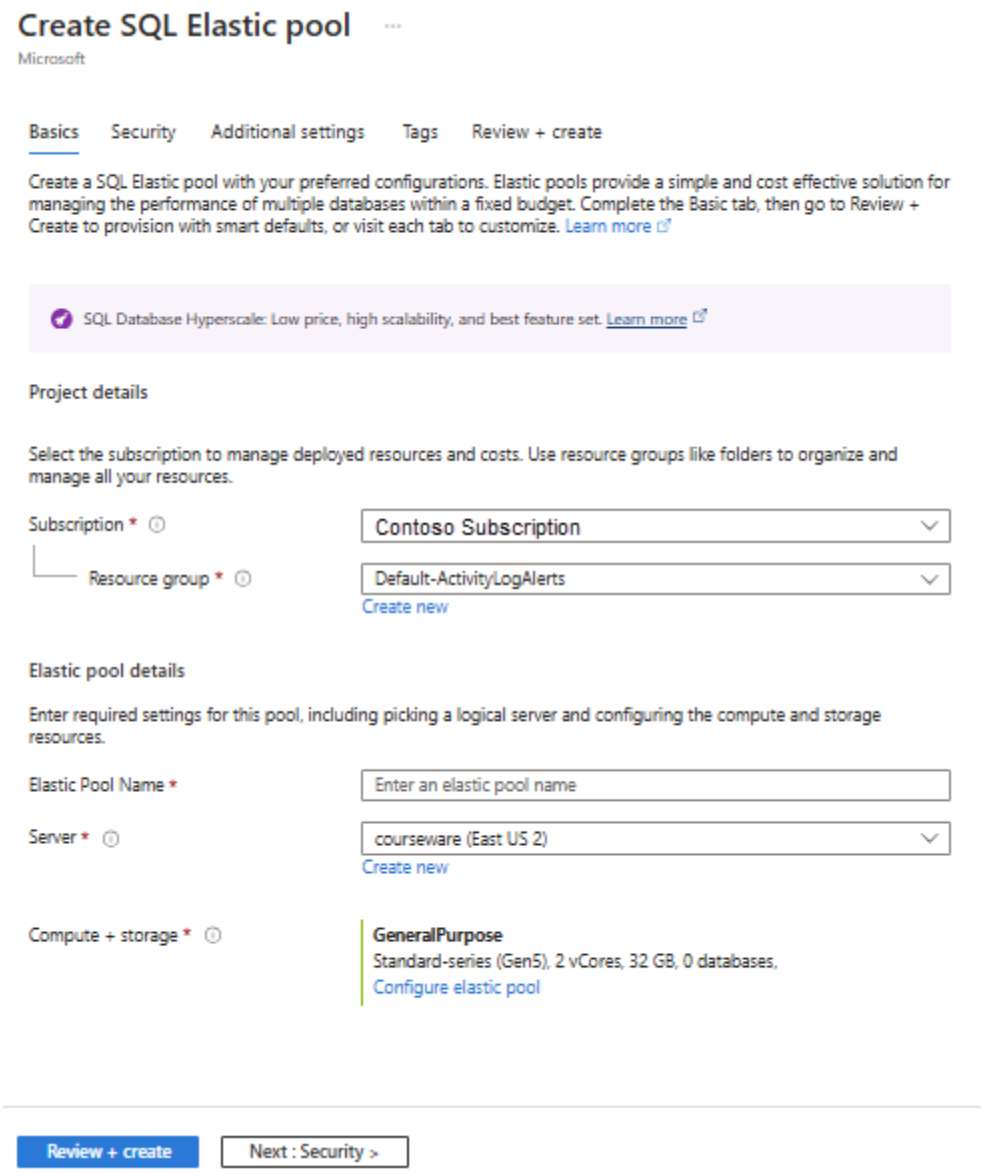
- This script automates the creation of an Azure resource group and the deployment of a SQL logical server within it.
- It prompts the user for a project name, Azure location, and SQL server administrator credentials, then creates a resource group using the provided details.
- Finally, it deploys a SQL logical server to the resource group using a predefined ARM template from a specified URI, and pauses execution until the user presses ENTER.

Deploy SQL database elastic pool

- Elastic pools are a deployment option in which you purchase Azure compute resources (CPU, memory, and storage) that is then shared among multiple databases defined as belonging to the same pool.
- An easy comparison to an on-premises SQL Server is that an elastic pool is like a SQL Server instance that has multiple user databases.
- By using elastic pools, you can easily manage pool resources while at the same time potentially saving costs.
- Elastic pools also facilitate easy scalability up to the set limits such that if a single database within the pool needs resources due to an unpredictable workload, the resources are there.
- If the entire pool needs extra resources, a simple slider option within the Azure portal facilitates scaling the elastic pool up or down.

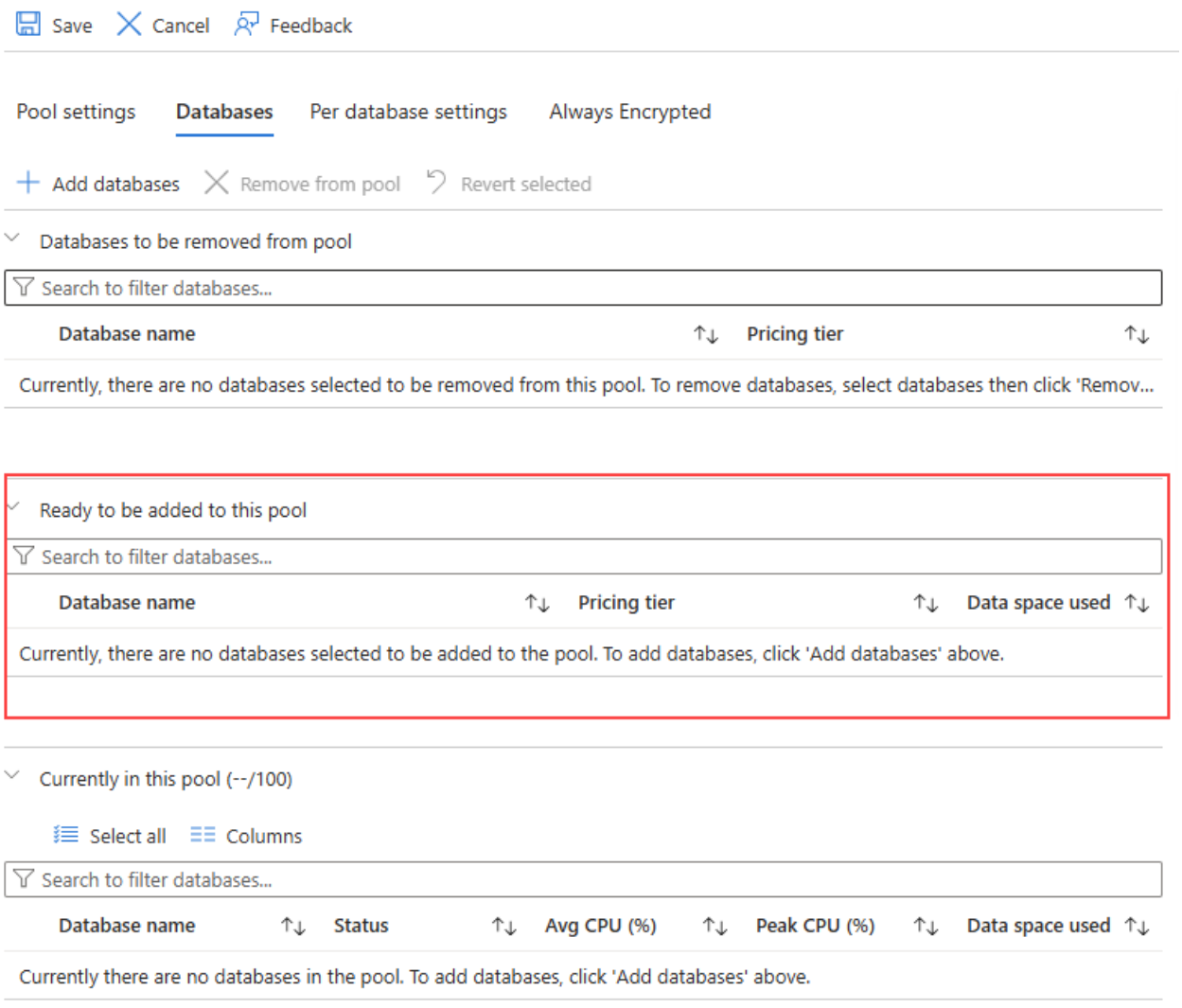
Creating new elastic pools

- Using the Azure portal, search for “SQL elastic pools”.
- Then select Create to open the Create SQL Elastic pool page.

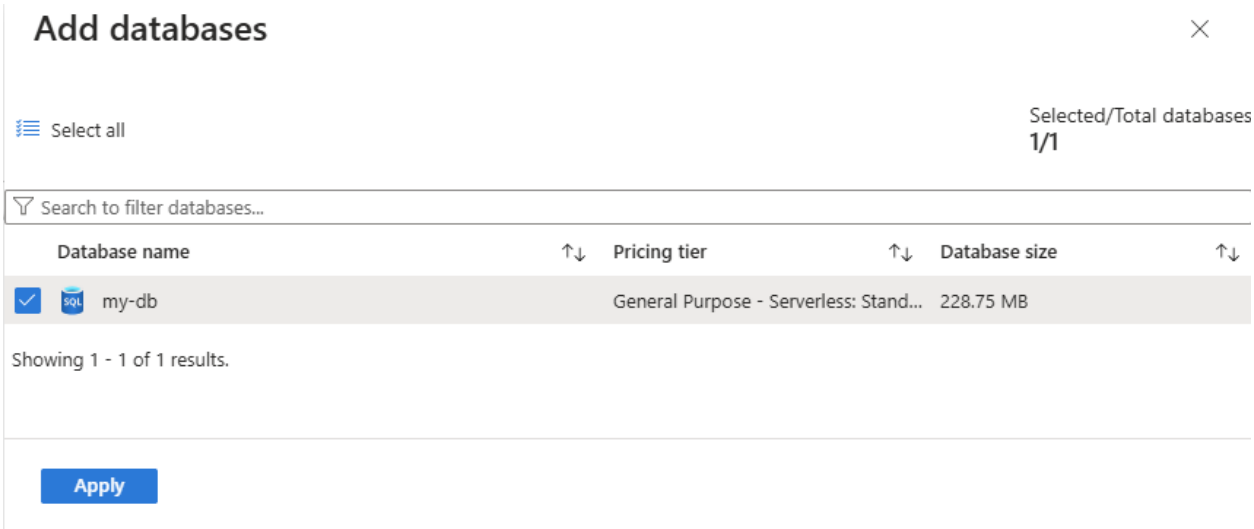


Adding a database to an existing pool

- Using the Azure portal, locate the pool to which you're adding a database.



• Select + Add databases to add your database to the pool, then select Apply.

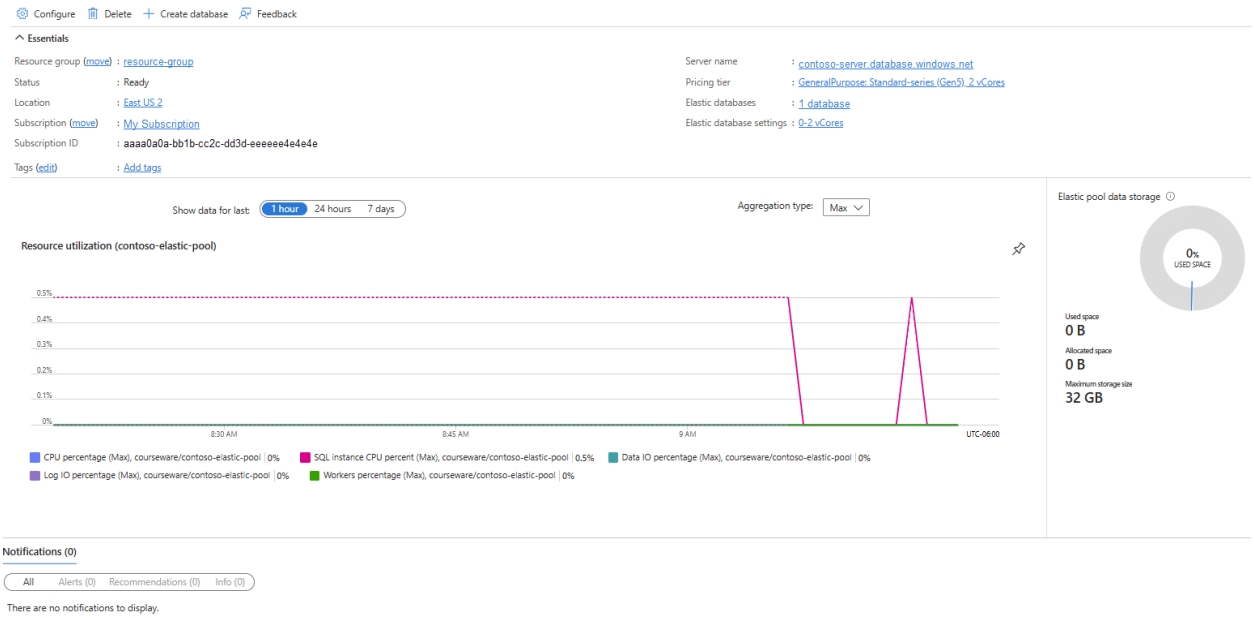


• Your database selection is then added to the Ready to be added to this pool section.
• Select Save.

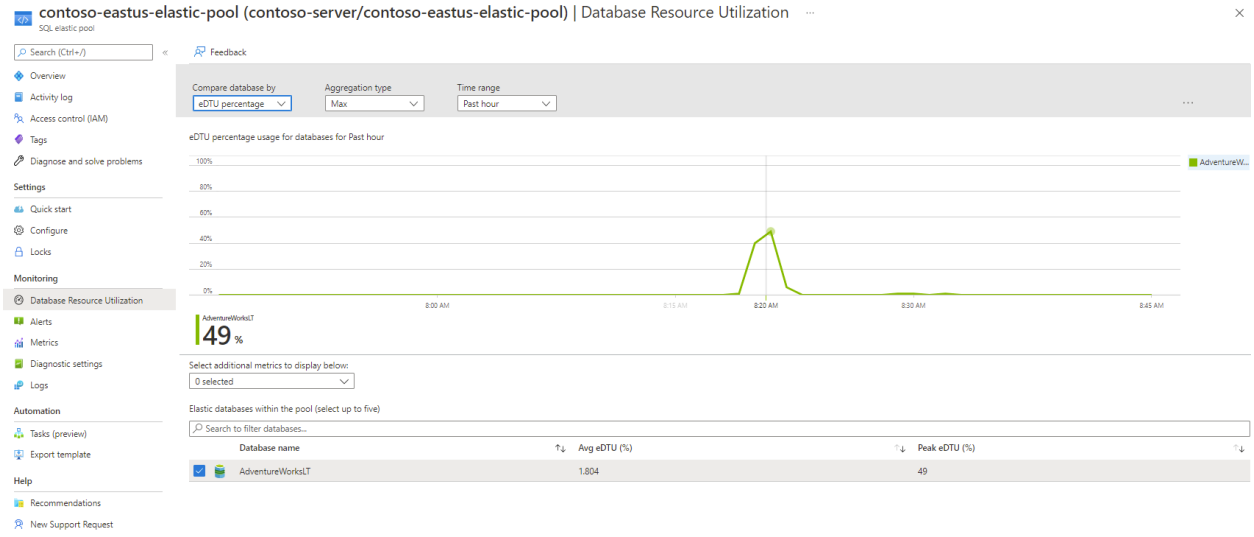
Managing pool resources

• The Azure portal provides comprehensive insights into the state and health of your elastic pool.

- You can monitor resource utilization and identify which database is consuming the most resources.
- This information is valuable for identifying performance issues or determining if a database isn't well-suited for the pool, especially if one database is using most of the resources.



- To adjust the resources allocated to your elastic pool, use the Configure option in the Settings section of the elastic pool management side menu.
- You can perform many changes after the creation of an elastic pool.
- Pool size, including DTUs, vCores, and storage size.
- Service tier.
- Resources per database.
- Databases included in the pool, by adding or removing them.
- Some changes, such as the minimum and maximum DTUs or vCores per database are performed online.
- You can also change the total size of the pool or add and remove databases as needed.
- Active connections are ended as the resizing completes.
- One of the most useful features is the ability to monitor database resource utilization.
- This feature provides an easy way to assess the performance of databases within the pool.



- An elastic pool is a good fit for multitenant databases where each tenant has its own copy of the database.

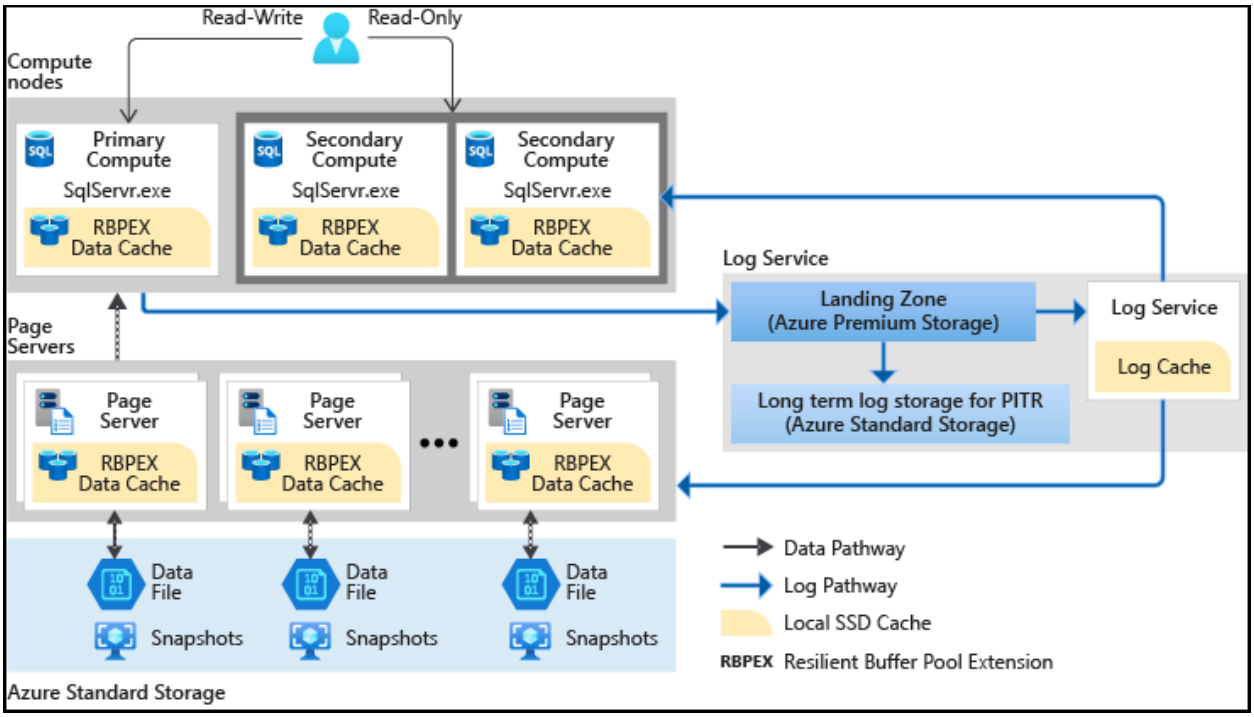
- Balance the workload across databases so as not to allow one database to monopolize all the pool’s resources.

Understand SQL database hyperscale

- Azure SQL Database was historically limited to 4 TB of storage per database due to physical infrastructure constraints.
- However, the Hyperscale service tier revolutionizes this by allowing databases to exceed 100 TB.
- Hyperscale uses horizontal scaling techniques to add compute nodes as data sizes grow.
- While the cost of Hyperscale is similar to Azure SQL Database, there's an extra per terabyte storage cost.
- It's important to note that once a database is converted to Hyperscale, it can't be reverted to a standard Azure SQL Database.
- Hyperscale is ideal for most business workloads, offering flexibility, and high performance with independently scalable compute and storage resources.
- It separates the query processing engine from the components providing long-term storage and durability, allowing storage capacity to scale smoothly as needed.
- The Hyperscale service tier, part of the vCore-based purchasing model, is the newest and most scalable option, significantly exceeding the limits of the General Purpose and Business Critical tiers.

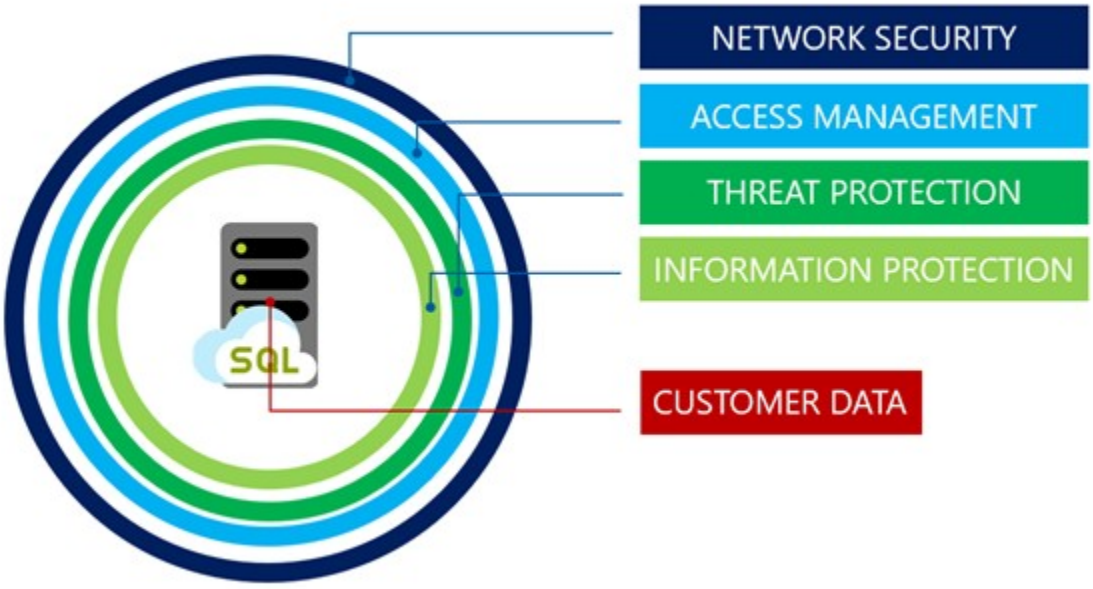
Benefits

- The Hyperscale service tier eliminates many of the practical limitations traditionally associated with cloud databases.
- The resources of a single node constrain most databases, but Hyperscale databases have no such restrictions.
- With its flexible storage architecture, storage expands as needed, and there's no predefined maximum size.
- You're billed only for the capacity you use.
- For read-intensive workloads, Hyperscale offers rapid scale-out by provisioning more replicas to handle read operations.
- Moreover, the time required for database backups or scaling operations is no longer dependent on the data volume.
- Hyperscale databases can be backed up instantly, and you can scale a database with tens of terabytes up or down in minutes.
- This flexibility ensures that your initial configuration choices don't constrain you.
- Additionally, Hyperscale provides fast database restores, completing in minutes rather than hours or days.
- Hyperscale provides rapid scalability based on your workload demand.
- Scaling Up/Down – You can increase or decrease the primary compute resources, such as CPU and memory, quickly and efficiently.
- Since the storage is shared, these scaling operations aren't dependent on the database's data volume.
- Scaling In/Out – You can create more compute replicas to handle read requests, effectively offloading the reading workload from the primary compute.
- These replicas also serve as hot standby, ready to take over if there's a primary compute failure.
- Provisioning more compute replicas is a quick, online operation.
- To connect to these read-only replicas, set the ApplicationIntent argument in your connection string to ReadOnly.
- Connections with the ReadOnly application intent are automatically routed to one of the read-only compute replicas.



Security considerations

- Security for the Hyperscale service tier offers the same robust capabilities as other Azure SQL Database tiers.
- It employs a layered defense-in-depth approach, providing comprehensive protection from the outermost layers inward.



- Network Security is the first layer of defense, utilizing IP firewall rules to control access based on the originating IP address.
- Additionally, Virtual Network firewall rules enable communication from selected subnets within a virtual network.
- Access Management is provided through the following authentication methods to verify user identity:

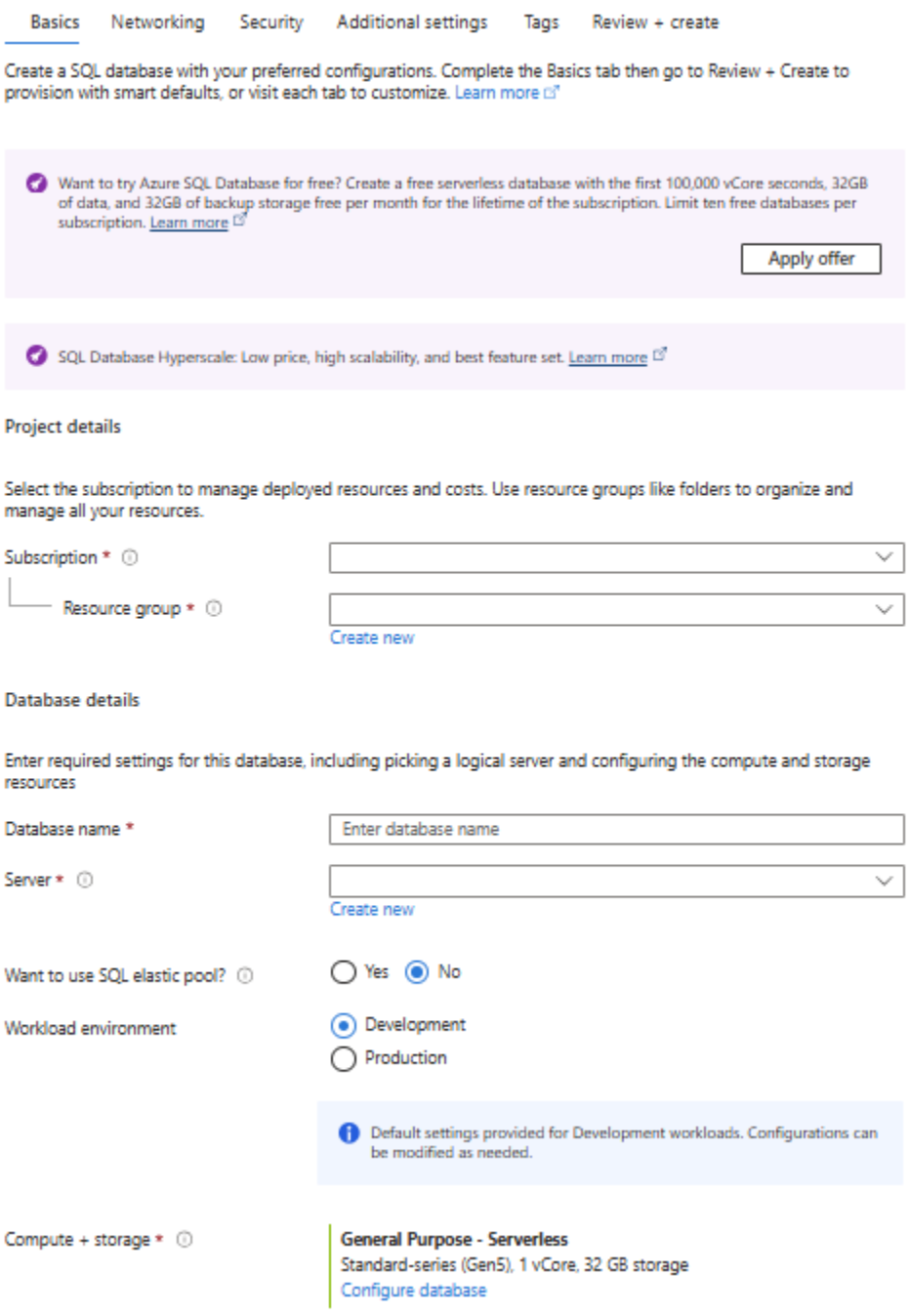
- **SQL Authentication**
- **Microsoft Entra authentication**
- Windows Authentication for Microsoft Entra principals
- Azure SQL Database Hyperscale also supports Row-Level Security (RLS), allowing customers to control access to specific rows in a database table based on user characteristics, such as group membership or execution context.
- Threat Protection includes robust auditing and threat detection capabilities.
- SQL Database and SQL Managed Instance auditing track database activities and help maintain compliance with security standards by recording events to an audit log in a customer-owned Azure storage account.
- Advanced Threat Protection analyzes your logs to detect unusual behavior and potential threats to your databases.
- It generates alerts for suspicious activities such as SQL injection, potential data infiltration, brute force attacks, and anomalies in access patterns that can indicate privilege escalations or the use of breached credentials.
- Information Protection is provided in the following ways:
- Transport Layer Security (Encryption-in-transit)
- Transparent Data Encryption (Encryption-at-rest)
- **Key management with Azure Key Vault**
- **Always Encrypted (Encryption-in-use)**
- **Dynamic data masking**

Performance considerations

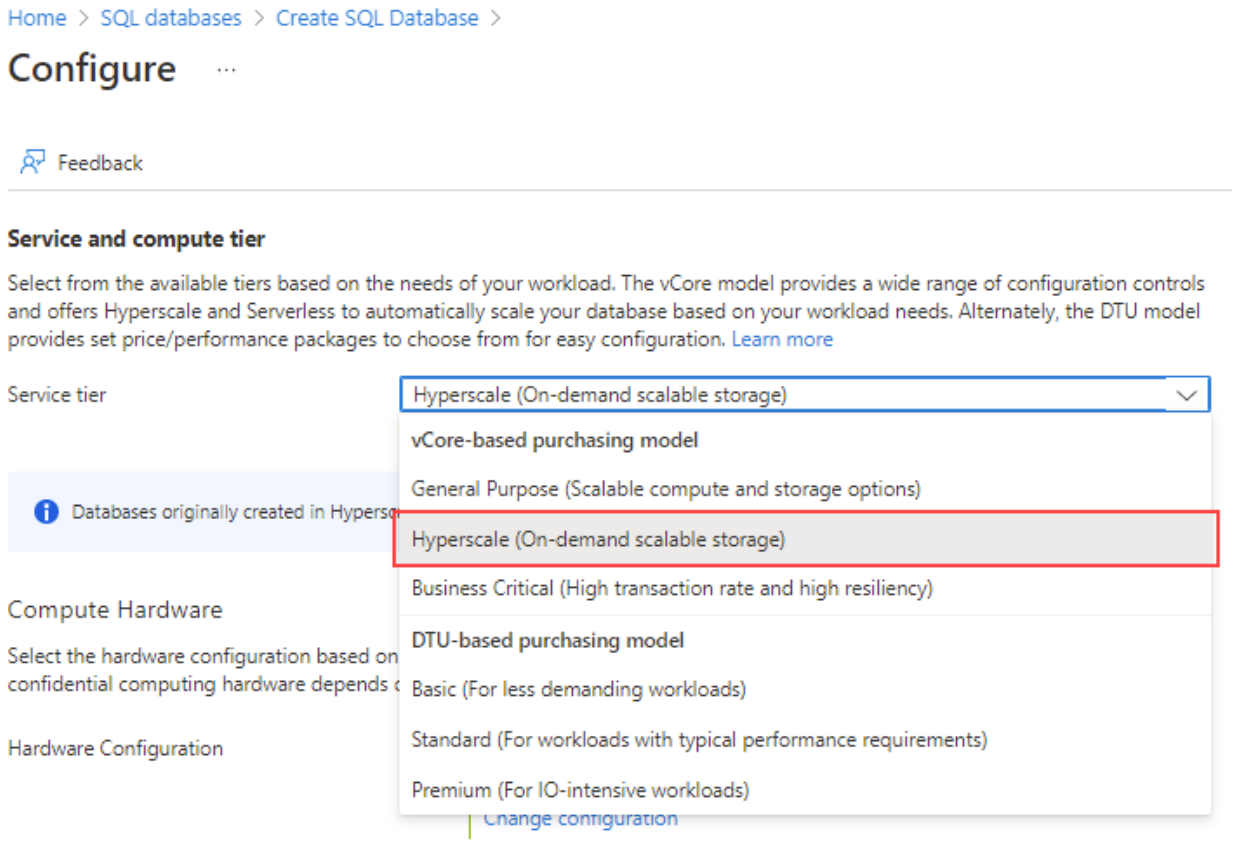
- The Hyperscale service tier is designed for customers with large on-premises SQL Server databases who want to modernize by moving to the cloud, and for those already using Azure SQL Database who need to significantly expand their database capacity.
- It's also ideal for customers seeking high performance and scalability.
- Key performance capabilities of Hyperscale include:
- Nearly instantaneous database backups using file snapshots stored in Azure Blob storage, without affecting compute resources.
- Fast database restores based on file snapshots, completing in minutes rather than hours or days, regardless of data size.
- Enhanced overall performance due to higher transaction log throughput and faster transaction commit times, irrespective of data volumes.
- Rapid scale-out by provisioning one or more read-only replicas to offload read workloads and serve as hot standby.
- Rapid scale-up, allowing you to quickly increase compute resources to handle heavy workloads and scale them back down when not needed.

Deploying Azure SQL Database Hyperscale

- To deploy an Azure SQL Database with the Hyperscale tier, follow the same process as deploying a regular SQL database, with the following differences:
- Under Compute + storage, select the Configure database link.



- For Service tier, select Hyperscale.



- Review the hardware configurations available and select the most appropriate configuration for your database.
- Optionally, review the other tabs to make adjustments if needed.
- On the Review + create tab, select Create.

Examine SQL managed instance

- While many organizations initially migrate to Azure using IaaS offerings, the platform as a service (PaaS) provides extra benefits.
- With PaaS, the service handles installing and patching SQL Server, so you no longer need to perform these tasks.
- Consistency checks, backups, security, and performance tools are also included as part of the managed service.
- Azure SQL Managed Instance is a fully functional SQL Server instance that is nearly 100% compatible with your on-premises ecosystem.
- It includes features like SQL Agent, access to tempdb, cross-database queries, and common language runtime (CLR).
- This service uses the same infrastructure as Azure SQL Database and offers all the benefits of PaaS, such as automatic backups, automatic patching, and built-in high availability.

Azure SQL Managed Instance capabilities

- Azure SQL Managed Instance offers a seamless migration path for existing applications by enabling restores from on-premises backups.
- Unlike Azure SQL Database, which is designed for single database structures, SQL Managed Instance provides a full SQL Server instance, supporting up to 100 databases and granting access to system databases.
- It also includes features not available in Azure SQL Database, such as cross-database queries, common language runtime (CLR), and the use of SQL Agent through the msdb system database.
- When creating an Azure SQL Managed Instance, you can choose between two service tiers: Business Critical and General Purpose.

- These tiers align with the Azure SQL Database vCore model, as SQL Managed Instances are purchased using this model.
- The primary differences between the two tiers are that Business Critical includes In-Memory OLTP and provides a readable secondary, features not available in the General Purpose tier.
- Both tiers offer high availability, with the Business Critical tier using Always On Availability Groups for higher resilience.
- Additionally, both tiers allow for independent configuration of storage and compute resources.

Managed Instance link

- The Link feature provides hybrid capabilities by replicating databases from SQL Server instances to Azure SQL Managed Instance.
- It uses distributed availability groups, part of the Always On availability group technology, to replicate data.
- Transaction log records are replicated as part of these distributed availability groups.
- Transaction log records on the primary instance can't be truncated until they're replicated to the secondary instance.
- Regular transaction log backups help reduce the risk of running out of space on your primary instance.
- The Link feature can also be used as a hybrid disaster recovery solution, allowing you to fail over your SQL Server databases hosted anywhere to a database running on SQL Managed Instance.
- Additionally, you can use the Link feature to provide a read-only secondary database in SQL Managed Instance, offloading intensive read-only operations.
- For more information about how to configure link feature for Azure SQL Managed Instance, see Prepare environment for link feature - Azure SQL Managed Instance.

Instance pool

- Instance pools provide a cost-efficient way to migrate smaller SQL Server instances to the cloud.
- Instead of consolidating smaller databases into a larger managed instance, which requires extra governance and security planning, instance pools allow you to pre-provision resources based on your total migration needs and requirements.
- The instance pool feature offers a fast deployment time of up to 10 minutes, making it ideal for scenarios where deployment duration is crucial.
- Additionally, all instances in a pool share the same virtual machine, and the total IP allocation is independent of the number of instances deployed.
- To learn how to deploy an instance pool for SQL Managed Instance, see Deploy Azure SQL Managed Instance to an instance pool .

High availability and Disaster recovery

- Azure SQL Managed Instance, as a PaaS service, inherently provides high availability.
- A standalone SQL Managed Instance offers a 99.99% Service Level Agreement (SLA), ensuring a maximum of 52.60 minutes of downtime per year.
- The architecture mirrors that of Azure SQL Database: the General Purpose tier uses storage replication for availability, while the Business Critical tier employs multiple replicas for enhanced resilience.
- Azure SQL Managed Instance offers auto-failover groups for disaster recovery, protecting the entire managed instance and all its databases.
- This feature asynchronously replicates data from the primary Azure SQL Managed Instance to a secondary instance, but it's currently limited to the paired Azure region of the primary instance, with only one replica allowed.
- Similar to Azure SQL Database, auto-failover groups provide read-write and read-only listener endpoints, simplifying connection string management.
- If there's a failover, the application connection strings are automatically routed to the appropriate instance.
- However, these endpoints follow slightly different formats.
- For SQL Managed Instance, the read/write listener endpoint uses <fog-name>.dns-zone.database.windows.net, and the read-only listener endpoint uses <fog-name>.secondary.dns-zone.database.windows.net.

- For Azure SQL Database, the corresponding endpoints are <fog-name>.database.windows.net and <fog-name>.secondary.database.windows.net.
- Both the primary and secondary managed instances must be within the same DNS zone.
- This ensures that the same multi-domain certificate can be used for client connection authentication between the instances in the failover group.
- You can specify a “DNS Zone Partner” through the Azure portal, PowerShell, or Azure CLI.

Backups

- Automatic backups are configured by default for Azure SQL Managed Instance.
- A key difference between Azure SQL Managed Instance and Azure SQL Database is that with SQL Managed Instance, you can manually create a copy-only backup of a database.
- These backups must be stored to a URL, as local storage access isn't permitted.
- Additionally, you can configure long-term retention (LTR) to retain automatic backups for up to 10 years in geo-redundant Azure Blob storage.
- Database backups follow the same schedule as Azure SQL Database, and these schedules can't be adjusted.
- **Full – Once a week**
- **Differential – Every 12 hours**
- Transaction Log – Every 5-10 minutes depending on transaction log usage
- Restoring a database to an Azure SQL Managed Instance is similar to the process with Azure SQL Database.
- You can use Azure portal, PowerShell, or Azure CLI.
- To restore from one instance to another, both instances must reside within the same Azure subscription and region.
- Additionally, you can't restore the entire managed instance, only individual databases within the SQL Managed Instance.
- As with Azure SQL Database, you can't restore over an existing database.
- You need to drop or rename the existing database before restoring it from backup.
- Since SQL Managed Instance is a fully functional SQL Server instance, you can execute a RESTORE command, which isn't possible with Azure SQL Database.
- However, as a PaaS service, there are some limitations:
- You must restore from a URL endpoint, as local drives aren't accessible.
- You can use the following options (in addition to specifying the database):
- **FILELISTONLY**
- **HEADERONLY**
- **LABELONLY**
- **VERIFYONLY**
- Backup files containing multiple log files can't be restored
- Backup files containing multiple backup sets can't be restored
- Backups containing In-Memory/FILESTREAM can't be restored
- By default, databases in a managed instance are encrypted using Transparent Data Encryption (TDE) with a Microsoft-managed key.
- To take a user-initiated copy-only backup, you must disable TDE for the specific database.
- If a database is encrypted, you can restore it, but you need access to either the certificate or asymmetric key used for encryption.
- Without these, you can't restore the database to a SQL Managed Instance.
- To learn the new features for Azure SQL Managed Instance, see What's new in Azure SQL Managed Instance?.

Exercise: Deploy an Azure SQL Database

- Now it's your chance to deploy an Azure SQL Database.
- In this exercise, you'll learn how to configure basic resources needed to deploy an Azure SQL Database with a Virtual Network Endpoint.

Note

- To complete this exercise, you'll need an Azure subscription.
- Launch the exercise and follow the instructions.

Launch Exercise

Summary

- **1 minute**
- In this module, you learned about the platform as a service options for SQL Server on Azure.
- Azure SQL Database is a flexible platform, well suited for software as service applications and has many options for large databases, multiple databases, or development environments that don't need to be online 24x7.
- Azure SQL Managed Instance is a PaaS version of SQL Server that allows you to easily move your on-premises environments into a managed service.
- SQL Managed Instance also supports many server-level capabilities that aren't available with Azure SQL Database.
- Now that you've reviewed this module, you should be able to:
- Understand PaaS provisioning and deployment options
- Understand elastic pools and hyperscale features
- **Examine SQL Managed Instances**
- Migrate SQL Server workloads to Azure SQL Database

Introduction

- Azure SQL Database is a Platform as a Service (PaaS) database engine that provides a complete development and deployment environment in the cloud, which can be used for simple cloud-based applications and for advanced enterprise applications.
- Migrating to SQL Database allows you to modernize your application by taking advantage of its PaaS capabilities.
- This enables you to eliminate dependencies on technical components that are scoped at the instance level, such as SQL Agent jobs.
- Azure SQL Database offers a low-maintenance solution that can be an excellent choice for certain workloads.
- You may have specific requirements that are better suited for Azure SQL Database rather than Azure SQL Managed Instance in the following scenarios:
- You need to simplify deployment for databases with intermittent, unpredictable usage.
- New databases without usage history where compute sizing is difficult or not possible to estimate prior to deployment.
- The complexity of deployment and development is a concern.
- Your storage requirements are higher than what Azure SQL Managed Instance offers, and database consolidation isn't an option.

- The process of migrating a SQL Server database running on an Azure virtual machine to Azure SQL Database is similar to the steps we'll learn in this module.

Note

- Before proceeding, it is important to ensure that you have reviewed the Assess SQL Server databases for migration to Azure SQL.
- This module will introduce you to the assessment tools and help you discover new features in the target SQL Server platform that your database can benefit from after an upgrade.

Use case scenario

- Throughout this module, we're using an example scenario to explain key data migration concepts.
- Suppose you work for a company that builds bikes and bicycle parts.
- You have several legacy database servers that you want to upgrade, including a product database, a parts stock database, and a human resources database.
- You also want to move from a capital expenditure model to an operational expenditure model, and benefit from the scalability and availability of Azure services.
- You plan to migrate your SQL Server databases to Azure SQL Database.
- Your board of directors has asked you to plan the migration project and has made you responsible for the execution of the migration tasks.
- You'll learn how to migrate SQL Server databases to Azure SQL Database.
- You'll begin by exploring the pre-migration considerations you need to take into account before a migration, and how to create an Azure SQL database.
- You'll then explore the different methods for offline and online migrations, and look at ways to move data to Azure SQL Database.

- Choose the right Azure SQL Database feature
- In our bicycle manufacturing scenario, you've already identified and profiled the databases that you want to migrate to Azure SQL Database.
- Now, you want to plan the migration, considering data recoverability, disaster recovery, security, and other implementation details.
- You'd like to know the tools and features available to support with the migration process to Azure SQL Database.

Benefits of Azure SQL Database

- The following summarize the benefits of deploying single and elastic pool databases:

Category	Feature
Backup and recovery	Automatic backup
	Point-in-time restore
	Backup retention 7 days+
	Long-term backup retention stores backups for up to 10 years
High availability	99.99% availability guarantee
	Built-in availability with three secondary replicas
	Zone redundancy via Azure availability zones
Disaster recovery	Geo-restore of database backups

Category	Feature
	Active-geo replication between Azure regions
Service scalability	Dynamic scale-up and scale-down
	Scale out with multiple shards
	Share compute resources between databases using elastic pools
Security	Support for Microsoft Entra authentication
	Cloud-only security features such as Advanced Threat Protection
	Transparent data encryption (TDE) enabled by default
	Support for dynamic and static data masking, row-level security, and Always Encrypted
	Firewall allowlist
Licensing	DTU purchasing model for predictive costing
	vCore purchasing model, enabling storage to be scaled independently of compute
	Combine the vCore purchasing model with Azure Hybrid Benefit for SQL Server to realize cost savings of up to 30 percent

Tip

- To review the benefits of migrating to Azure SQL Database and the features available, please refer to Deploy PaaS solutions with Azure SQL module.

Exclusive features of Azure SQL Database

- Some features are supported in Azure SQL Database that aren't available in other Azure SQL offerings:

Feature	Definition
Hyperscale	Cloud-native architecture that allows for independently scalable compute and storage, providing greater flexibility and resources than other tiers.
Auto-scale	With serverless compute tier
Automatic tuning (indexes)	This built-in feature automatically identifies and creates indexes that can improve the performance of your workload. It also verifies that query performance has improved and removes unused or duplicate indexes.
Elastic query	Allows you to run T-SQL queries that bridge multiple databases in SQL Database. This feature is useful for applications using three- and four-part names that can't be changed.
Elastic jobs	The elastic job feature is the SQL Server Agent replacement for Azure SQL Database. To some extent, elastic job is equivalent to the Multi Server Administration feature available on SQL Server instance.
Query Performance Insights (QPI)	This tool helps find the queries to optimize to improve overall workload performance and efficiently use the resource that you're paying for.

Important

- To understand additional feature differences between SQL Database, SQL Server, and Azure SQL Managed Instance, as well as the differences among different Azure SQL Database options, see SQL Database features.

Migration options supported

- There are two modes of migration to Azure SQL Database: Online and Offline.
- The online mode has minimal or no downtime, while the offline mode experiences downtime during the migration process.

Tool	Migration mode
Azure Database Migration Service	**Offline**
Transactional replication	**Online**
Azure Migrate	**Offline**
Import Export Wizard/BACPAC	**Offline**
Bulk copy (bcp utility)	**Offline**
Azure Data Factory	**Offline**

* Can have a higher performance impact, depending on the workload.

Note

- We recommend that you use the Azure Database Migration Service for large migrations and enhanced overall experience.

Migration performance

- Consider the following recommendations when migrating to Azure SQL Database:
- Monitor data file I/O and latency on the source, and mitigate any bottlenecks.
- Scale up the target Azure SQL database to Business Critical Gen5 8 vCore or use the Hyperscale service tier to minimize latency for log files.
- Ensure that your network bandwidth can accommodate the maximum log ingestion rate.
- Choose the highest service tier and compute size for maximum transfer performance, and scale down after migration.
- Minimize the distance between BACPAC files and the destination data center.
- Disable auto update and auto create statistics during migration.
- Partition tables and indexes, drop indexed views, and recreate them after migration.
- Consider migrating rarely queried historical data to a separate database in Azure SQL Database, and query it using elastic queries.

Retry application connections

- When migrating to Azure SQL Database, it's important to anticipate occasional transient failures when connecting to the database resource, and implement a proper retry logic method.
- Setting a maximum number of retries before the program terminates is also important.
- We recommend waiting for 5 seconds at a minimum on your first retry.
- Each subsequential retry should increase the delay exponentially, up to a maximum of 60 seconds.

Note

- If a SELECT statement fails with a transient error for SQL Database, don't directly retry it.
- Instead, retry the SELECT statement in a new connection.

- To learn more about the connection retry principals, see Troubleshoot transient connection errors in SQL Database and SQL Managed Instance.

- Use Azure Database Migration Service to migrate to Azure SQL Database
- If you can afford to take the database offline while you migrate to Azure, you have a few tools you can use.
- In our bicycle manufacturing scenario, the HR database is considered business-critical but is rarely used at weekends.
- You've planned to execute an offline migration between Friday evening and Monday morning, but you want to assess the best migration method.
- It's assumed that all pre-migration checks have been done with Azure Migrate.
- This process ensures that feature and compatibility issues are addressed.
- Migrate using Azure Database Migration Service with Azure CLI
- Azure Database Migration Service is a fully managed service designed to enable seamless migrations from multiple database sources to Azure data platforms with minimal downtime.
- You can use Azure CLI or PowerShell to automate database migrations, making it ideal for migrating databases at scale.
- The Azure CLI az datamigration extension provides commands to create and manage database migrations to Azure SQL Database.
- This approach is particularly useful for:
- Automating migrations as part of CI/CD pipelines
- **Migrating multiple databases at scale**
- **Scripting repeatable migration processes**

Prerequisites

- Before starting the migration, ensure you have:
- Azure CLI installed with the datamigration extension
- Azure Database Migration Service created in your subscription
- Target Azure SQL Database provisioned with the schema already deployed
- Self-hosted integration runtime configured for connectivity to your source SQL Server
- To install the Azure CLI datamigration extension, run:

Azure CLI

```
az extension add --name datamigration
```

Create the Database Migration Service

- First, create an Azure Database Migration Service to orchestrate your migration activities:

Azure CLI

```
# Create the Azure Database Migration Service
az datamigration sql-service create \
  --resource-group "<YourResourceGroup>" \            # Name of your Azure resource group
  --sql-migration-service-name "<YourMigrationService>" \  # Name for the migration service
  --location "<YourLocation>"                         # Azure region (e.g., eastus2, westus)
```

Migrate the database schema

- Before migrating data, you need to migrate the database schema from the source to the target.
- Use the az datamigration sql-server-schema command:

Azure CLI

```
# Migrate schema from source to target database
az datamigration sql-server-schema \
  --action "MigrateSchema" \
```

```
  --src-sql-connection-str "Server=<YourSourceServer>;Initial Catalog=<YourSourceDB>;User
ID=<YourSourceUser>;Password=<YourSourcePassword>" \
  --tgt-sql-connection-str "Server=<YourTargetServer>.database.windows.net;Initial Catalog=<YourTargetDB>;User
ID=<YourTargetUser>;Password=<YourTargetPassword>"
```

Start the database migration

- Create a new database migration to copy data from the source to target:

Azure CLI

```
# Create a database migration to Azure SQL Database
az datamigration sql-db create \
  --resource-group "<YourResourceGroup>" \          # Name of your Azure resource group
  --sqldb-instance-name "<YourTargetServer>" \      # Name of the target Azure SQL Database server
  --target-db-name "<YourTargetDB>" \               # Name of the target database
  --source-database-name "<YourSourceDB>" \         # Name of the source database
  --source-sql-connection authentication="SqlAuthentication" \
```

- data-source="<YourSourceServer>" \ # Source SQL Server hostname or IP
- user-name="<YourSourceUser>" \ # Source SQL Server username

```
  password="<YourSourcePassword>" \ # Source SQL Server password
  encrypt-connection=true \
  trust-server-certificate=true \
  --target-sql-connection authentication="SqlAuthentication" \
```

- data-source="<YourTargetServer>.database.windows.net" \ # Target Azure SQL Database server
- user-name="<YourTargetUser>" \ # Target database username

```
  password="<YourTargetPassword>" \ # Target database password
  encrypt-connection=true \
  trust-server-certificate=true \
  --scope "/subscriptions/<YourSubscription>/resourceGroups/<YourResourceGroup>/providers/Microsoft.Sql/servers/
<YourTargetServer>" \
  --migration-service "/subscriptions/<YourSubscription>/resourceGroups/<YourResourceGroup>/providers/
Microsoft.DataMigration/sqlMigrationServices/<YourMigrationService>"
```

Migrate specific tables

- To migrate only specific tables, use the --table-list parameter:

Azure CLI

```
# Create a database migration for specific tables
az datamigration sql-db create \
  --resource-group "<YourResourceGroup>" \
  --sqldb-instance-name "<YourTargetServer>" \
  --target-db-name "<YourTargetDB>" \
  --source-database-name "<YourSourceDB>" \
  --source-sql-connection authentication="SqlAuthentication" \
  data-source="<YourSourceServer>" \
  user-name="<YourSourceUser>" \
  password="<YourSourcePassword>" \
  encrypt-connection=true \
  trust-server-certificate=true \
  --target-sql-connection authentication="SqlAuthentication" \
  data-source="<YourTargetServer>.database.windows.net" \
```

```
  user-name="<YourTargetUser>" \
  password="<YourTargetPassword>" \
  encrypt-connection=true \
  trust-server-certificate=true \
  --table-list "[Person].[Person]" "[Person].[EmailAddress]" "[Sales].[Customer]" \  # Specify tables to migrate
  --scope "/subscriptions/<YourSubscription>/resourceGroups/<YourResourceGroup>/providers/Microsoft.Sql/servers/
<YourTargetServer>" \
  --migration-service "/subscriptions/<YourSubscription>/resourceGroups/<YourResourceGroup>/providers/
Microsoft.DataMigration/sqlMigrationServices/<YourMigrationService>"
```

- The Database Migration Service optimizes the migration process by skipping empty tables, even if you select them.

Migration status

- There are a few statuses that keep you updated on the progress of the migration.
- Preparing for copy: The service is in the process of disabling autostats, triggers, and indexes in the target table.
- Copying: The data copy from the source database to the target database is in progress.
- Copy finished: Data copy is finished, and the service is waiting on other tables to finish copying.
- Rebuilding indexes: The service is rebuilding indexes on target tables.
- Succeeded: All data is copied and the indexes are rebuilt.

Monitor migration using Azure CLI

- You can check the status of your migration using the az datamigration sql-db show command:

Azure CLI

```
# Check the status of the database migration
az datamigration sql-db show \
  --resource-group "<YourResourceGroup>" \
  --sqldb-instance-name "<YourTargetServer>" \
  --target-db-name "<YourTargetDB>" \
  --expand "MigrationStatusDetails"               # Include detailed migration status
```

- This command returns detailed information about the migration, including the current status and any errors encountered.

Wait for migration completion

- You can use the wait command to pause script execution until the migration completes:

Azure CLI

```
# Wait for the migration to complete before continuing
az datamigration sql-db wait \
  --resource-group "<YourResourceGroup>" \
  --sqldb-instance-name "<YourTargetServer>" \
  --target-db-name "<YourTargetDB>" \
  --created                                 # Wait until migration is created/completed
```

Cancel a migration

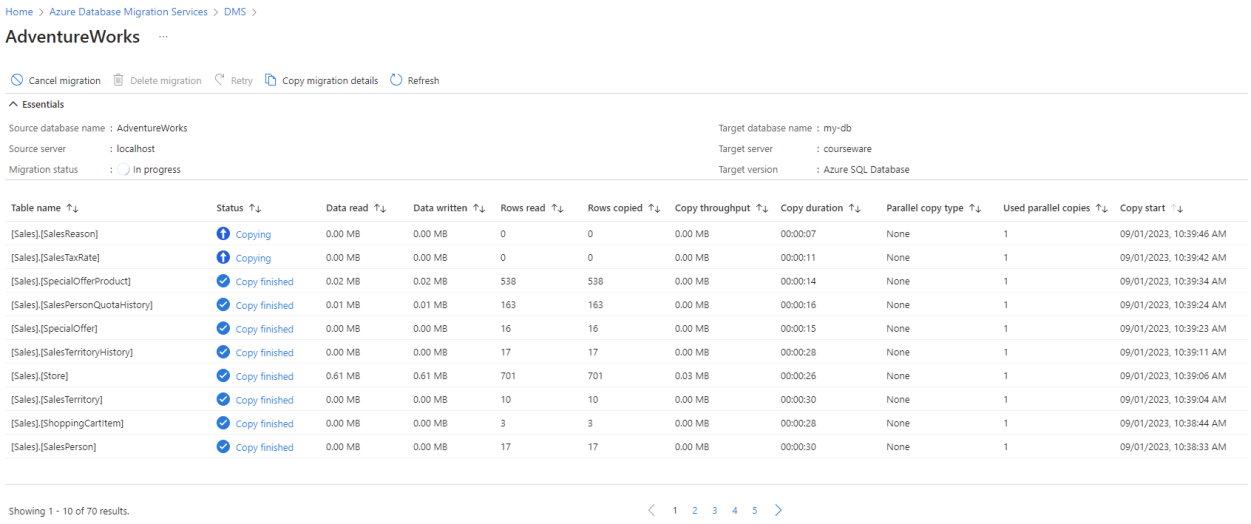
- If you need to stop an in-progress migration:

Azure CLI

```
# Cancel an in-progress migration
az datamigration sql-db cancel \
  --resource-group "<YourResourceGroup>" \
  --sqldb-instance-name "<YourTargetServer>" \
  --target-db-name "<YourTargetDB>" \
  --migration-operation-id "<YourMigrationOperationId>"  # ID from the migration operation
```

Monitor migration from the Azure portal

- You can also monitor the migration activity using Azure Database Migration Service in the Azure portal.
- To monitor your database migration, go to the Azure portal and find your instance of the Database Migration Service.
- Once you've located the service, you can view its instance overview.
- Select Monitor migrations to access detailed information about your ongoing database migration.



- After the migration status is Succeeded, navigate to the target server, and validate the target database.
- Check the database schema and data.

Performance considerations

- Migration speed heavily depends on the target Azure SQL Database SKU and the self-hosted Integration Runtime host.
- We strongly recommend that you scale up your Azure SQL Database compute resources before initiating the migration process for an optimal migration experience.
- When deciding on the server to install the self-hosted integration runtime, make sure this machine can handle the CPU and memory load of the data copy operation.
- Azure SQL Database migration can be slow with a large volume of tables due to the time Azure Data Factory (ADF) takes to start activities, even for small tables.
- Tables with large blob columns may fail to migrate due to timeout.
- We recommend up to 10 concurrent database migrations per self-hosted integration runtime on a single computer.
- Scale out the self-hosted runtime or create separate instances on different computers to increase the concurrent database migrations.

Migrate at scale using PowerShell

- You can also perform an offline migration of the database from SQL Server on-premises to an Azure SQL Database by using PowerShell.
- The following example migrates the AdventureWorks database to Azure SQL Database.

PowerShell

```
# Set up secure credentials for source and target connections
$sourcePass = ConvertTo-SecureString "<YourSourcePassword>" -AsPlainText -Force
$targetPass = ConvertTo-SecureString "<YourTargetPassword>" -AsPlainText -Force

# Start the database migration to Azure SQL Database
```

```
New-AzDataMigrationToSqlDb `
  -ResourceGroupName "<YourResourceGroup>" ` # Name of your Azure resource group
```

- -SqlDbInstanceName "<YourTargetServer>" ` # Name of the target Azure SQL Database server

```
  -Kind "SqlDb" `
  -TargetDbName "<YourTargetDB>" ` # Name of the target database
  -SourceDatabaseName "<YourSourceDB>" ` # Name of the source database
  -SourceSqlConnectionAuthentication SQLAuthentication `
  -SourceSqlConnectionDataSource "<YourSourceServer>" ` # Source SQL Server hostname or IP
  -SourceSqlConnectionUserName "<YourSourceUser>" ` # Source SQL Server username
  -SourceSqlConnectionPassword $sourcePass `
  -Scope "/subscriptions/<YourSubscription>/resourceGroups/<YourResourceGroup>/providers/Microsoft.Sql/servers/
  <YourTargetServer>" `
  -TargetSqlConnectionAuthentication SQLAuthentication `
  -TargetSqlConnectionDataSource "<YourTargetServer>.database.windows.net" `
  -TargetSqlConnectionUserName "<YourTargetUser>" ` # Target database username
  -TargetSqlConnectionPassword $targetPass `
```

- -MigrationService "/subscriptions/<YourSubscription>/resourceGroups/<YourResourceGroup>/providers/Microsoft.DataMigration/SqlMigrationServices/<YourMigrationService>"
- The following example migrates a subset of tables from the AdventureWorks database.

```
PowerShell
# Migrate specific tables from source to target database
New-AzDataMigrationToSqlDb `
  -ResourceGroupName "<YourResourceGroup>" `
  -SqlDbInstanceName "<YourTargetServer>" `
  -Kind "SqlDb" `
  -TargetDbName "<YourTargetDB>" `
  -SourceDatabaseName "<YourSourceDB>" `
  -SourceSqlConnectionAuthentication SQLAuthentication `
  -SourceSqlConnectionDataSource "<YourSourceServer>" `
  -SourceSqlConnectionUserName "<YourSourceUser>" `
  -SourceSqlConnectionPassword $sourcePass `
  -Scope "/subscriptions/<YourSubscription>/resourceGroups/<YourResourceGroup>/providers/Microsoft.Sql/servers/
  <YourTargetServer>" `
  -TargetSqlConnectionAuthentication SQLAuthentication `
  -TargetSqlConnectionDataSource "<YourTargetServer>.database.windows.net" `
  -TargetSqlConnectionUserName "<YourTargetUser>" `
  -TargetSqlConnectionPassword $targetPass `
```

- -TableList "[Person].[Person]", "[Person].[EmailAddress]" ` # Specify tables to migrate
- -MigrationService "/subscriptions/<YourSubscription>/resourceGroups/<YourResourceGroup>/providers/Microsoft.DataMigration/SqlMigrationServices/<YourMigrationService>"
- To learn more about database migration commands, refer to the following links: PowerShell module for data migration and Azure CLI for data migration.

Migrate to Azure SQL Database using BACPAC

- A SQL Server database can be imported into an Azure SQL database using a .bacpac file.

- A .bacpac file is a compressed file containing the metadata and data from the database.
- The data can be imported from the Azure Blob Storage or from a local storage in an on-premises environment.
- For optimal scale and performance in production environments, it's recommended to use the SQLPackage utility.
- Running multiple SqlPackage commands in parallel for subsets of tables can significantly accelerate import/export operations.
- Import from a BACPAC file in the Azure portal
- You can follow these steps to import a .bacpac file in Azure SQL Database.
- To import from a BACPAC file into a new single database using the Azure portal, open the appropriate database server page and then, on the toolbar, select Import database.
- Select the storage account and container for the BACPAC file, and then select the BACPAC file from which to import.
- Specify the new database size (usually the same as the origin) and provide the destination SQL Server credentials, and then select OK.
- To monitor an import's progress, open the database server page and, under Settings, select Import/Export history.
- When successful, the import has a Completed status.
- Additionally, you can also use SqlPackage to import a BACPAC file as it's faster than using the Azure portal.
- The following command imports the AdventureWorks2019 database from local storage to an Azure SQL Database server called <server-name>.
- It creates a new database called myMigratedDatabase with a Premium service tier and a P6 service objective.
- Change these values as appropriate for your environment.

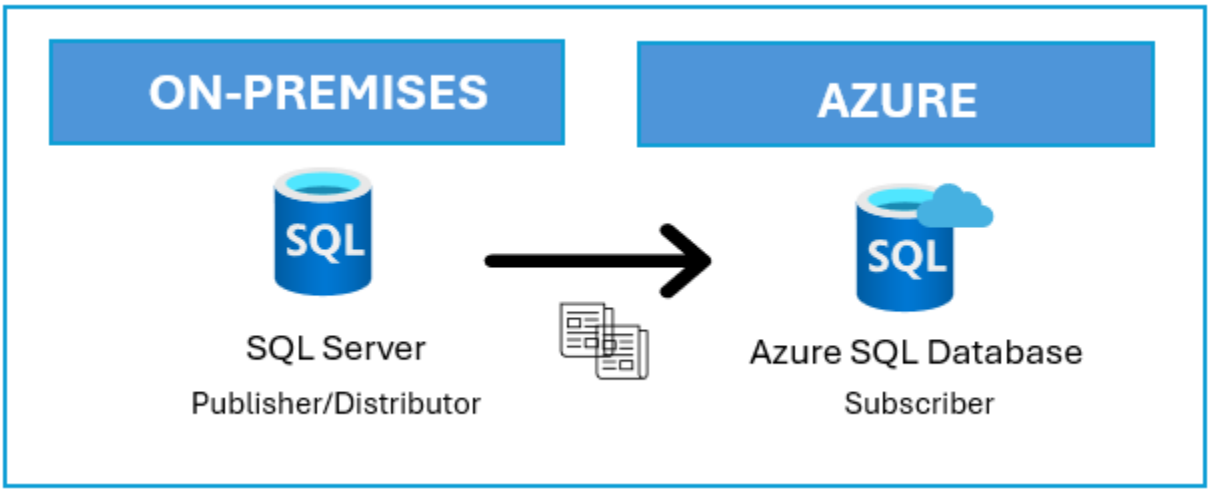
Console

- SqlPackage.exe /a:import /tcs:"Data Source=<server-name>.database.windows.net;Initial Catalog=myMigratedDatabase;User Id=<your_server_admin_account_user_id>;Password=<your_server_admin_account_password>" /sf:AdventureWorks2019.bacpac /p:DatabaseEdition=Premium /p:DatabaseServiceObjective=P6

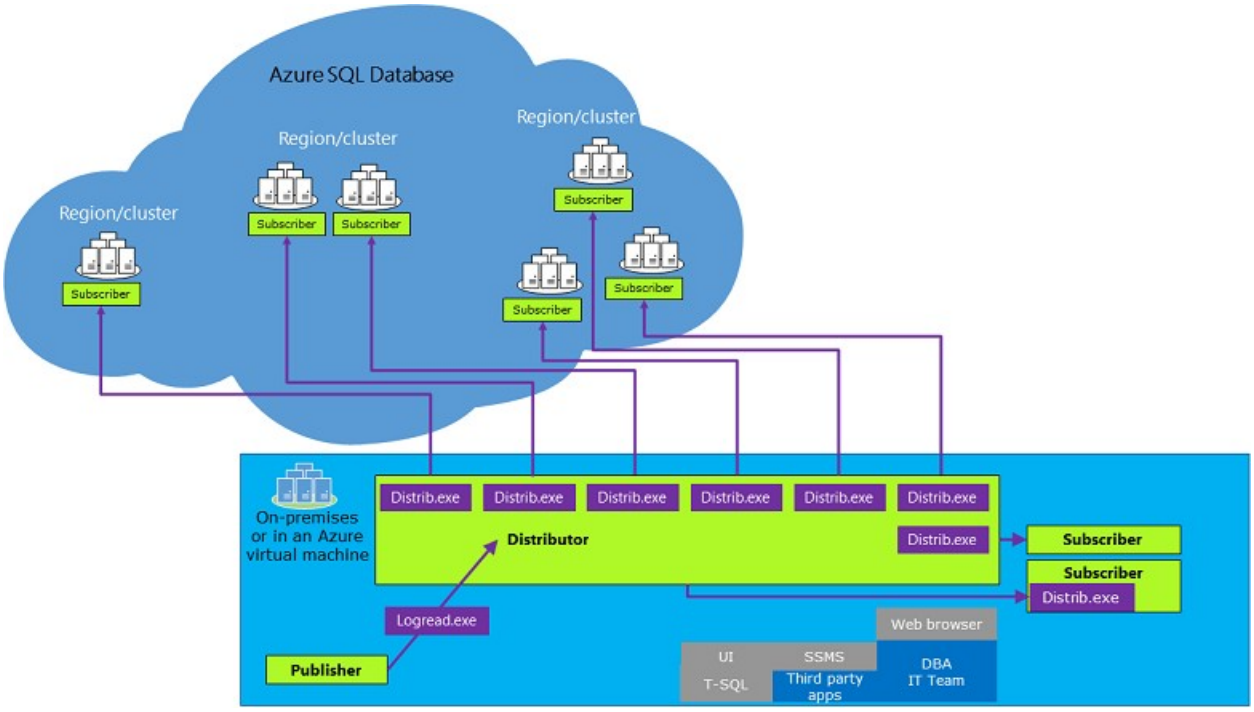
Tip

- To increase the speed of the import process, you can scale your database to a higher service tier and compute size, providing more and faster resources.
- Once the import is complete, you can scale down to your desired service tier and compute size.

- Use an online method to migrate to Azure SQL Database
- If you need a database to remain online to users throughout the migration process, you can use transactional replication to move the data.
- Transactional replication is the only online method available for migrating to Azure SQL Database.
- In our bicycle manufacturer scenario, warehouses run on a 24 hour, 7 days a week basis, and there are no periods of inactivity.
- Your board of directors wants to be sure that the inventory database is constantly available, even during the migration to Azure SQL Database.



- What is transactional replication?
- Transactional replication is a way to move data between continuously connected database servers.
- The process begins with a snapshot of the publication database objects and data.
- Once the initial snapshot is taken, any subsequent changes to the data or schema at the Publisher are typically delivered to Azure SQL Database in near real-time as they occur.



- Azure SQL Database supports both transactional and snapshot replication as a push subscriber.
- That means Azure SQL Database can receive and apply changes from a publisher using either a transactional or snapshot replication.
- The publisher and/or distributor can be an instance of SQL Server that is either running on-premises, on an Azure virtual machine in the cloud, or as an Azure SQL Managed Instance.
- You can configure transactional replication through SQL Server Management Studio, or by executing Transact-SQL statements on the publisher.
- Transactional replication can't be configured from the Azure portal.
- Transactional replication requires the following components:

Role	Definition
Publisher	A database instance that hosts the data to be replicated (source).
Subscriber	Receives the data being replicated by the *Publisher* (target).
Distributor	Collects changes in the articles from a *Publisher* and distributes them to the *Subscribers*.
Article	A database object; for example, a table that's included in the *Publication*.
Publication	A collection of one or more articles from the database being replicated.
Subscription	A request from a *Subscriber* for a *Publication*.

Set up a transactional replication

- Follow the steps below to migrate the table [Person].[Person] from AdventureWorks database to Azure SQL Database with no downtime.
- Transactional replication can only use SQL Server authentication logins to connect to Azure SQL Database.

Parameter	Definition
@distributor	Source instance name.
@publisher	Source instance name.
@subscriber	Azure SQL Database in the format: <server>.database.windows.net. The Azure SQL Database must exist before running the script.
@dbname	Database name at the source.
@publisher_login	SQL user with required permissions at the source.
@publisher_password	Password for the SQL user.
@destination_db	Database name at the destination.
@subscriber_login	SQL user with required permissions at the destination.
@subscriber_password	Password for the SQL user.
@working_directory	Replication working directory, change this location as appropriate.

- Adjust the parameters above according to your own environment when running the script.

Create the distributor

- The following script creates the distributor database, distributor publishers, and the agents.

SQL

```
USE [master]
GO

EXEC sp_adddistributor @distributor = N'CONTOSO-SRV', @password = N''
GO

EXEC sp_adddistributiondb
```

@database = N'distribution',

- @data_folder = N'C:\Program Files\Microsoft SQL Server\MSSQL16.MSSQLSERVER\MSSQL\Data',

@data_file = N'distribution.MDF',

@data_file_size = 13,

- @log_folder = N'C:\Program Files\Microsoft SQL Server\MSSQL16.MSSQLSERVER\MSSQL\Data',

@log_file = N'distribution.LDF',

@log_file_size = 9,

@min_distretention = 0,

@max_distretention = 72,

@history_retention = 48,

@deletebatchsize_xact = 5000,

@deletebatchsize_cmd = 2000,

@security_mode = 1

GO

```
-- Adding the distribution publishers
exec sp_adddistpublisher
```

@publisher = N'CONTOSO-SRV',

@distribution_db = N'distribution',

@security_mode = 1,

@working_directory = N'C:\REPL',

@trusted = N'false',

@thirdparty_flag = 0,

@publisher_type = N'MSSQLSERVER'

GO

```
exec sp_addsubscriber
```

- @subscriber = N'contoso.database.windows.net',

@type = 0,

- @description = N'Azure SQL Database (target)'

GO

```
-- Enabling the replication database
use master
exec sp_replicationdboption
```

@dbname = N'AdventureWorks',

@optname = N'publish',

@value = N'true'

GO

--Adds a Log Reader agent for the AdventureWorks database.

- exec [AdventureWorks].sys.sp_addlogreader_agent

@publisher_security_mode = 1

GO

--Adds a Queue Reader agent for the distributor.

- exec [AdventureWorks].sys.sp_addqreader_agent

@frompublisher = 1

GO

Create the transactional publication

- The following script creates the transactional publication of the AdventureWorks database from the publisher.

SQL

```
USE [AdventureWorks]
GO

EXEC sp_addpublication
```

@publication = N'REPL-AdventureWorks',

- @description = N'Transactional publication of database ''AdventureWorks'' from Publisher ''CONTOSO-SRV''.',

```
@sync_method = N'concurrent',
@retention = 0,
@allow_push = N'true',
@allow_pull = N'true',
@allow_anonymous = N'true',
@enabled_for_internet = N'false',
@snapshot_in_defaultfolder = N'false',
@alt_snapshot_folder = N'C:\REPL',
@compress_snapshot = N'true',
@ftp_port = 21,
@ftp_login = N'anonymous',
@allow_subscription_copy = N'false',
@add_to_active_directory = N'false',
@repl_freq = N'continuous',
@status = N'active',
@independent_agent = N'true',
@immediate_sync = N'true',
@allow_sync_tran = N'false',
@autogen_sync_procs = N'false',
@allow_queued_tran = N'false',
@allow_dts = N'false',
@replicate_ddl = 1,
@allow_initialize_from_backup = N'false',
@enabled_for_p2p = N'false',
@enabled_for_het_sub = N'false'
GO
```

exec sp_addpublication_snapshot

@publication = N'REPL-AdventureWorks',

@frequency_type = 1,

@frequency_interval = 0,

@frequency_relative_interval = 0,

@frequency_recurrence_factor = 0,

@frequency_subday = 0,

@frequency_subday_interval = 0,

@active_start_time_of_day = 0,

@active_end_time_of_day = 235959,

@active_start_date = 0,

@active_end_date = 0,

@publisher_security_mode = 0,

@publisher_login = N'sqladmin',

@publisher_password = N'<pwd>'

Create the article for the publication

- The following script creates the article for the [Person].[Person] table.

SQL

```
USE [AdventureWorks]
GO

EXEC sp_addarticle
```

@publication = N'REPL-AdventureWorks',

@article = N'Person',

@source_owner = N'Person',

@source_object = N'Person',

@type = N'logbased',

@description = N'',

@creation_script = N'',

@pre_creation_cmd = N'drop',

@schema_option = 0x000000000803509F,

@identityrangemanagementoption = N'none',

@destination_table = N'Person',

@destination_owner = N'Person',

@status = 24,

@vertical_partition = N'false',

- @ins_cmd = N'CALL [sp_MSins_PersonPerson]',

- @del_cmd = N'CALL [sp_MSdel_PersonPerson]',
- @upd_cmd = N'SCALL [sp_MSupd_PersonPerson]'

GO

- Create the subscription and the subscription agent
- The following script creates the push subscription to the Azure SQL Database subscriber.

SQL

```
USE [AdventureWorks]
GO

EXEC sp_addsubscription
```

@publication = N'REPL-AdventureWorks',

- @subscriber = N'contoso.database.windows.net',

@destination_db = N'my-db',

@subscription_type = N'Push',

@sync_type = N'automatic',

@article = N'all',

@update_mode = N'read only',

@subscriber_type = 0

```
exec sp_addpushsubscription_agent
```

@publication = N'REPL-AdventureWorks',

- @subscriber = N'contoso.database.windows.net',

@subscriber_db = N'my-db',

@job_login = null,

@job_password = null,

@subscriber_security_mode = 0,

@subscriber_login = N'sqladmin',

@subscriber_password = '<pwd>',

@frequency_type = 64,

@frequency_interval = 1,

@frequency_relative_interval = 1,

@frequency_recurrence_factor = 0,

@frequency_subday = 4,

@frequency_subday_interval = 5,

@active_start_time_of_day = 0,

@active_end_time_of_day = 235959,

@active_start_date = 0,

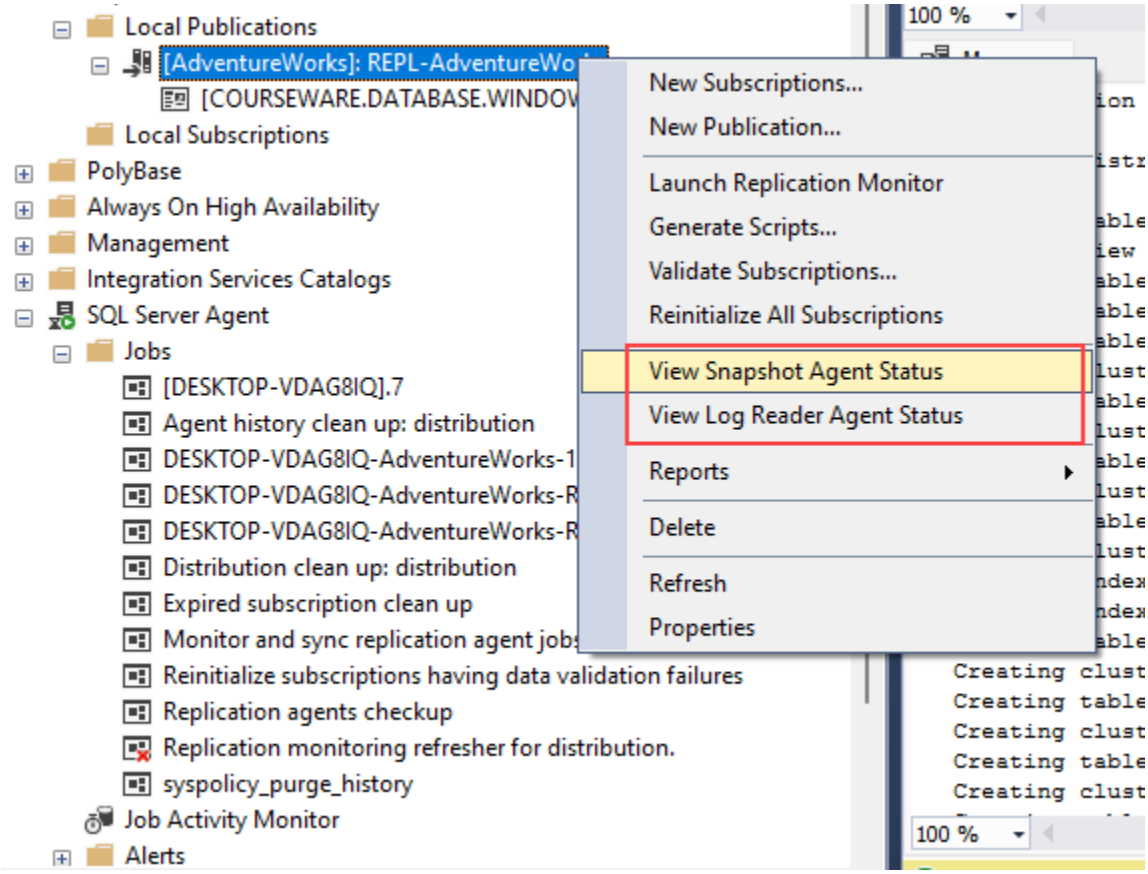
@active_end_date = 0,

@dts_package_location = N'Distributor'

GO

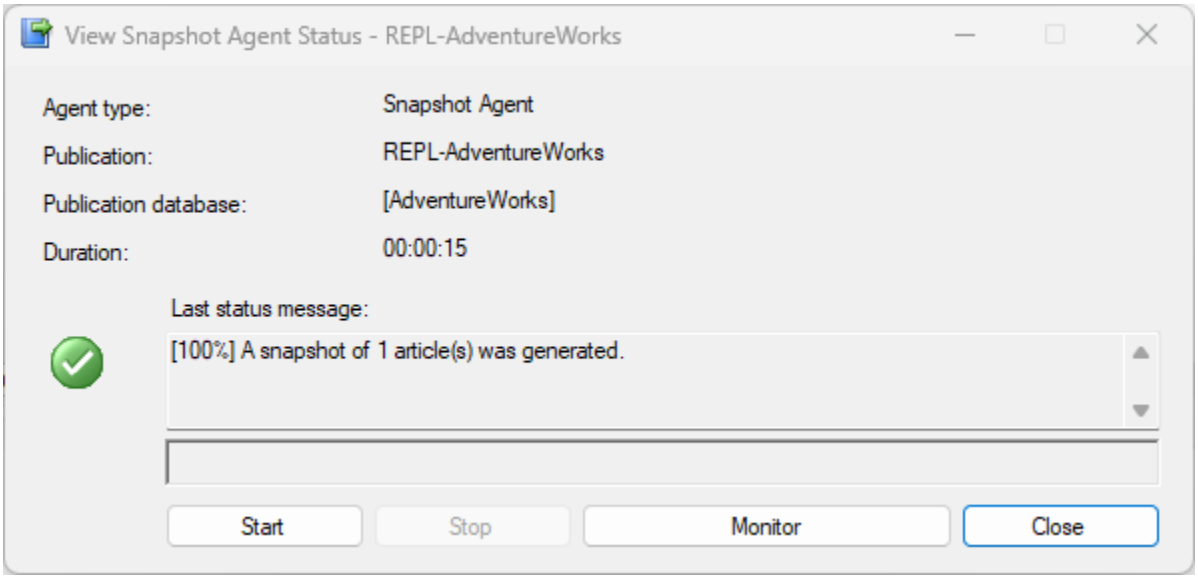
Initiate and monitor replication

- Replication management and monitoring are not supported from Azure SQL Database.
- Instead, perform these activities from SQL Server.
- To initiate replication, start the snapshot job, log reader job, and distributor job.
- You can monitor the Snapshot Agent and Log Reader Agent by right-clicking on the publication and selecting the appropriate option.
- If the agents are not running, start them.

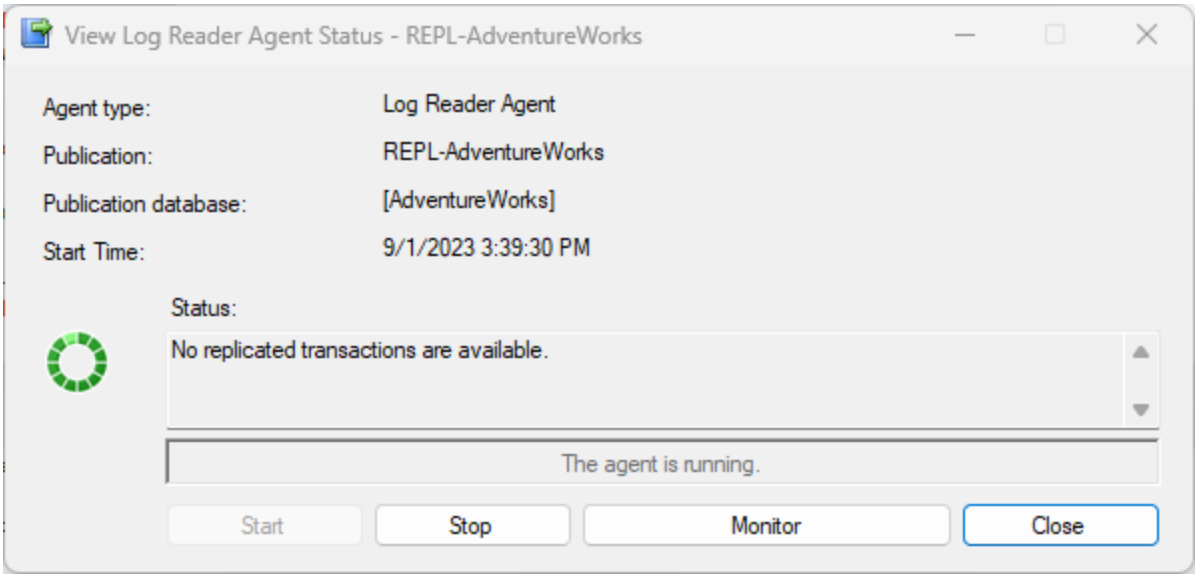


- To view the synchronization status, right-click on the subscription, select View Synchronization Status, and then start the agent.
- If you encounter any error messages, check the agent jobs history on the SQL Server Agent.
- If the agents are running as expected, you should see the following results.

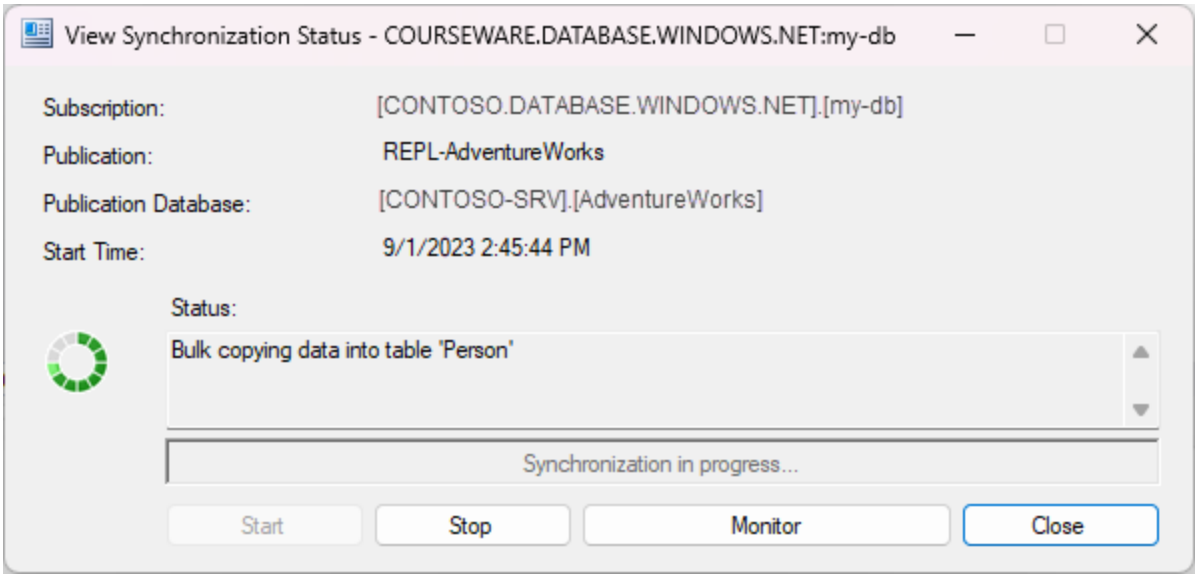
Snapshot Agent:



Log Reader Agent:



Synchronization Status:



- After the data is fully replicated to Azure SQL Database, you can direct the connections to the subscriber database, and then stop and remove the replication.
- To learn more about supported configurations, see Replication to Azure SQL Database.

Move data to Azure SQL Database

- While there are methods available for migrating an entire schema and its data, there are also cases where only a subset of the database is needed.
- Fortunately, many of the methods we've seen support partial data migration, and we'll learn about a few others.
- In our bicycle manufacturer scenario, suppose the company has an on-premises SQL Server database that contains several years' worth of sales, customer, and product data.
- The company wants to migrate to an Azure SQL Database to take advantage of the scalability and flexibility of the cloud.
- However, they only need to migrate the customer and product tables, as they want to keep their sales data on-premises for security reasons.

Bulk copy

- The bcp utility allows bulk exporting of data from a SQL Server table into a data file and vice versa.
- The utility is versatile, enabling data transfer between SQL Server and other programs or databases.
- Understanding the schema and data types of the table is essential for using the bcp command effectively, unless a preexisting format file is available.

Azure Data Factory

- You can use Azure Data Factory for data migration rather than entire database migration.
- Azure Data Factory can migrate and transform data from source SQL Server databases, and it's commonly used for business intelligence workloads (BI).

Exercise - Migrate a SQL Server database to Azure SQL Database

- Now it's your chance to migrate a SQL Server database using a few of the tools and features available.
- In this exercise, you learn how to migrate a SQL Server database to Azure SQL Database.

Note

- To complete this exercise, you need an Azure subscription.
- Launch the exercise and follow the instructions.

Launch Exercise

Summary

- In our bike manufacturing scenario, you created and executed a plan to migrate your servers to Azure SQL Database using both offline and online methods.
- Now, with the migration complete, your users have the best availability and scalability for their data and you're confident that you can respond to future changes in demand quickly.
- The method chosen to migrate the database is typically dependent on how much time the SQL Server databases can be offline.
- For additional reading, you can refer to the following resources:
- Migration overview: SQL Server to Azure SQL Database

Azure Database Migration ServiceAzure Database Migration Service

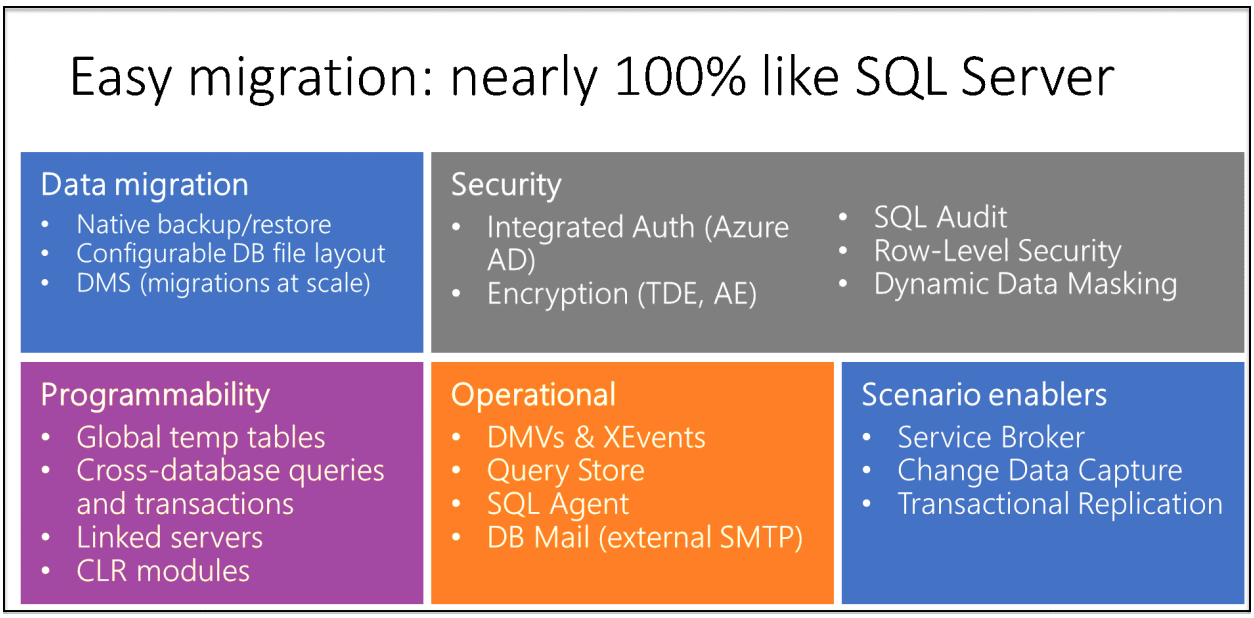
- Transactional Replication with Azure SQL Database

- Migrate SQL Server workloads to Azure SQL Managed Instance

Introduction

- Azure SQL managed instances make it easy and quick to lift and shift a database from on-premises into the cloud.
- Suppose you work for a sports clothing retail company.
- Your organization has been moving many of its systems into Azure to realize improved cost-of-ownership and better availability.
- So far, you haven't considered migrating your business-critical databases to Azure because you've heard that databases often need modification to work in Azure SQL.
- You want to evaluate Azure's database hosting solutions, choose the best one to host your database, and do the migration.

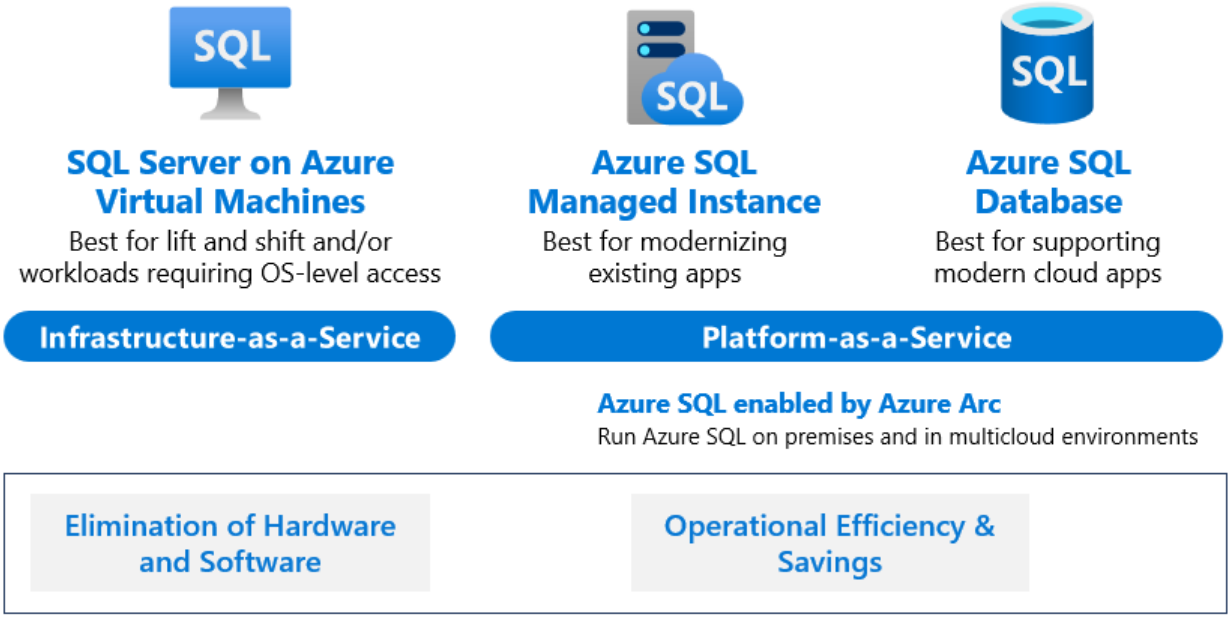
- You'd prefer to migrate with as little database administrator and developer time as possible to keep the project budget low.



- In this module, you'll learn about Azure SQL Managed Instance, how it differs from other database solutions available in Azure, and how to choose the best cloud database solution for your project.

Evaluate migration scenarios

- Azure SQL managed instance is designed to make it easy to host existing databases in the cloud by providing almost 100 percent compatibility with on-premises versions of SQL Server.
- In your sports clothing company, you have a database that stores the product details for your entire catalog.
- The website uses the database to display product details to customers, by the sales representatives' smartphone apps to keep them informed about the catalog, and by a data analysis solution to populate product dimensions in a data cube.
- The database is considered business-critical by the board of directors.
- You've been asked to migrate this database to the cloud so the systems that depend on it need as little modification as possible.
- You want to evaluate Azure SQL Managed Instance for this project.
- What is Azure SQL Managed Instance?
- The Azure SQL platform as a service (PaaS) family includes the Azure SQL Database and Azure SQL Managed Instance.
- The goal of Azure SQL Managed Instance is to provide SQL Server applications with a fully managed PaaS experience in the Azure cloud.



- Azure SQL Managed Instance is designed to enable a lift and shift solution for customers.
- The managed instance looks to bring applications, databases, and supporting technologies to Azure PaaS.
- Previously, without SQL Managed Instance, migration scenarios where an organization's application required access to any technology outside of the database (for example SQL Agent jobs, cross database joins, and SQL Server Integration Services) would be prevented from moving to the cloud.
- The only way a DBA or developer could migrate an on-premises application would be to employ one of the following approaches:
- Move the database and supporting technologies to an infrastructure as a service (IaaS) model.
- Rewrite the application with a fully PaaS model on Azure SQL Database, with extra development to address migration blockers.
- The decision to migrate applications to Azure often hinges on whether an organization has the resources to adapt their application to Azure's PaaS model and manage the application code, as vendor support for modifications is typically limited.
- So, many opt for SQL Server on IaaS to use the full SQL Server experience without the need to overhaul existing applications.
- Despite Azure SQL Database's capabilities, the heavy dependence of many applications on technologies outside its scope presents challenges.
- However, SQL Managed Instance, code-named "cloud lifter," aims to overcome these hurdles, facilitating the migration to a SQL-based PaaS solution in Azure without needing application redesign.

Review key features

- The most important features of SQL Managed Instance include:

Key Features	Description
Backwards compatibility	Managed instance provides backward compatibility to SQL Server 2008 databases. Direct migration from SQL Server 2005 database servers is also supported, with the compatibility level for migrated SQL Server 2005 databases being updated to SQL Server 2008.
Easy lift and shift	Managed instance has close to 100 percent compatibility to SQL Server. This compatibility includes core SQL Server components, programmability enhancements, instance-scoped features, such as cross database joins, and management tools that most existing SQL-based applications need to function correctly.
Fully managed	PaaS benefits include removing the need for managing hardware and all the overhead that comes from doing physical maintenance on SQL Server servers. You also have the benefits of quickly scaling up and

Key Features	Description
PaaS	scaling down, and provisioning resources in the cloud. SQL Managed Instance is built on the SQL Server engine, so it's always up to date with the latest SQL features and functionality.
Security features	You can enable security features at the SQL Managed Instance level just as you do at the database level. These features include the Vulnerability Assessment and the Advance Threat Protection settings. Finally, at the managed instance level, you can configure Transparent Data Encryption (TDE) and whether you want to bring your own key (BYOK) for encryption.
Secure network isolation	One of the unique aspects of managed instance, network security isolation is where managed instance has complete security isolation from any other tenant in the Azure cloud. In a typical default deployment SQL endpoint, managed instance is solely exposed through a private IP address that only allows connectivity from private Azure networks or hybrid networks. For on-premises applications to connect to managed instance, you'd need an Azure ExpressRoute configuration or a VPN gateway.
Instance failover groups	An instance failover group is a set of databases managed by a single database server, or within a single managed instance, that can fail over as a unit to another region. You use instance failover groups when all or some of the primary databases have gone offline due to an outage in the primary region.

Migration options supported

- There are two modes of migration to Azure SQL Managed Instance: online and offline.
- The online mode has minimal or no downtime, while the offline mode experiences downtime during the migration process.
- Log Replay Service.
- It's an online migration option, and used when you need more control of your database migration project.
- Managed Instance link..
- The Managed Instance link, using distributed availability groups, securely extends your data estate by replicating data almost instantly (online) between any hosted SQL Server and Azure SQL Managed Instance, and vice versa.
- SQL Server migration in Azure Arc.
- When your SQL Server instance is enabled by Azure Arc, you can migrate databases to SQL Managed Instance directly from the Azure portal using either the Managed Instance link or Log Replay Service methods.
- Native backup and restore.
- Backup and restore are a simple migration method favored by many SQL Server professionals.
- It's the easiest migration option for customers who can provide full database backups to Azure Storage.
- Transactional replication.
- Transactional replication is a way to move data between continuously connected database servers.
- It’s best to be used for online or offline migration of large and complex databases.
- While most of the tools facilitate migration to Azure SQL Database as well, there are some that are exclusively supported by SQL Managed Instance.
- In the next units, we'll learn about a few of them in more detail.

Tip

- Learn more about how to design a SQL Server migration strategy.

Migrate using Azure Arc

- Azure Arc provides a streamlined pathway to migrate SQL Server databases to Azure SQL Managed Instance directly from the Azure portal.
- When your SQL Server instance is enabled by Azure Arc, you gain access to integrated assessment, target selection, and migration capabilities—all managed through a unified experience.

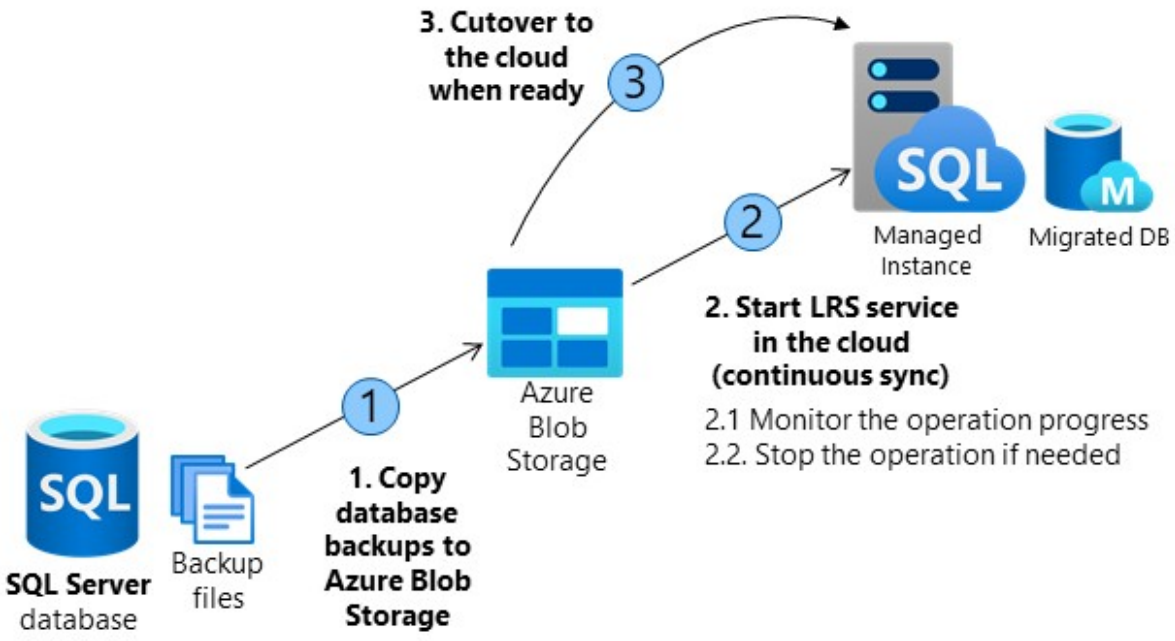
- In our bicycle manufacturing company scenario, the distribution team runs business-critical databases that require minimal downtime during migration.
- The team wants to use the Azure portal for a simplified migration experience while maintaining control over the migration method.
- Understand how to migrate SQL Server with Azure Arc
- When your SQL Server instance is enabled by Azure Arc, the Azure portal becomes your central hub for database migration to SQL Managed Instance.
- The migration process guides you through four key stages:
- Assess source instance - Evaluate your SQL Server instance to determine migration readiness
- Select target - Choose or create your SQL Managed Instance destination
- Migrate data - Execute the migration using your chosen method
- Monitor and cutover - Track progress and complete the migration
- Discovery of SQL Server instances and generation of readiness reports happen automatically every weekend, but you can also trigger assessments manually at any time.
- The assessment evaluates feature compatibility, identifies potential migration blockers, and provides right-sized recommendations for your target SQL Managed Instance configuration.
- Database migration through Azure Arc is available for SQL Server 2012 and later versions, with the specific migration method determining exact version requirements.

Benefits of Azure Arc for migration

- Azure Arc simplifies the migration journey by consolidating the entire process within the Azure portal.
- Rather than switching between multiple tools and interfaces, you can assess readiness, select or create your target instance, execute the migration, and monitor progress from a single unified experience.
- The automated assessment capabilities evaluate feature compatibility and identify potential migration blockers, while also providing right-sized recommendations for your target SQL Managed Instance configuration based on your workload characteristics.
- Microsoft Copilot is integrated into the Azure portal migration experience to assist you throughout the process.
- You can interactively chat with Copilot to get help comparing migration methods, understanding assessments, starting migrations, and monitoring progress—making it easier to choose and execute the right approach for your scenario.
- Additionally, Azure Arc provides visibility into your overall migration status, showing the number of databases migrated, migrations in progress, and your recommended target configuration.
- This centralized view helps you track progress across your data estate and ensures nothing is overlooked during large-scale migration projects.
- For detailed guidance on migrating SQL Server databases to Azure SQL Managed Instance using Azure Arc, see Migration to Azure SQL Managed Instance.

Use Log Replay Service (LRS) to migrate

- Log Replay Service (LRS) is a tool that enables custom migrations of databases from on-premises SQL Servers to SQL Managed Instance in the cloud.
- It uses log shipping technology and is useful in cases where more control is needed, when there's little tolerance for downtime, or when Azure Database Migration Service can't be used.



- LRS can be used directly with PowerShell, CLI cmdlets, or API, to manually build and orchestrate database migrations to SQL Managed Instance.
- Some of the reasons to consider using LRS include:
- More control over the database migration project
- Little tolerance for downtime on migration cutover
- Inability to install DMS executable in the environment
- **Lack of file access to database backups**
- Inability to open networking ports from the environment to Azure

Understand migration types

- There are two migration modes available for LRS.

Mode	Description	Recommended for	Backup Chain Availability
Autocomplete	Automatically finishes the migration when the last backup file is restored	Passive workloads	Entire backup chain must be available in advance
Continuous	Continuously scans for new backup files and restores them, allowing for data catch-up	Active workloads	Backup chain can be added during migration

- Regardless of the mode, plan to complete the migration within 30 days, as the LRS job will be automatically canceled after this time.

Secure the migration process

- To run LRS, you must have one of the following Azure role-based access control (RBAC) role: Subscription Owner, SQL Managed Instance Contributor, or a custom role with the permission Microsoft.Sql/managedInstances/databases/*.
- An Azure Blob Storage account is required and works as an intermediary storage for backup files between your SQL Server instance and your SQL Managed Instance.
- To use Azure Blob storage with a firewall, another configuration is required.
- You must add the SQL Managed Instance subnet to the storage account's virtual network firewall rules using MI subnet delegation and the Storage service endpoint.

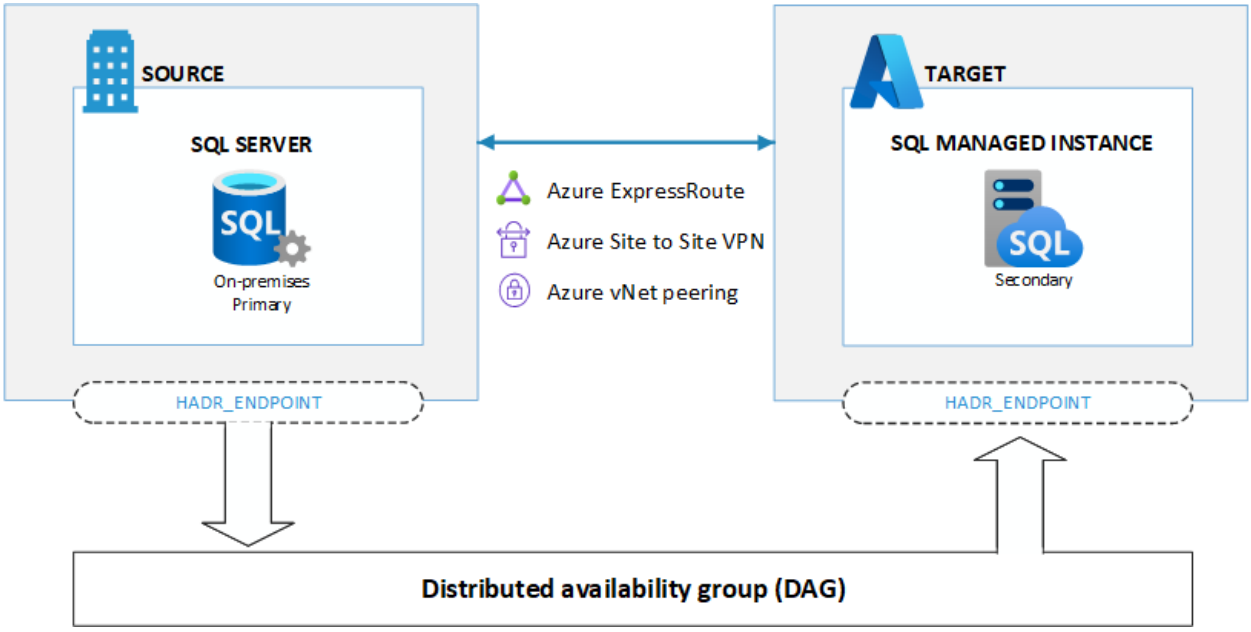
- Also, you can use either a SAS token or a managed identity to access your Azure Blob Storage account, but not both.

Improve backup and restore performance

- You can split full and differential backups into multiple files, instead of using a single file, to improve backup and restore performance.
- This is because multiple files can be read or written to in parallel, reducing the time it takes to complete the backup or restore operation.
- Also, enabling backup compression can help improve network transfer speeds.
- Compressed backups are smaller in size, which means they take less time to transfer over the network.
- This can be especially useful when transferring large backups to or from Azure.
- We strongly recommend enabling CHECKSUM for backups, even though it isn't required.
- SQL Managed Instance performs an integrity check on backups without CHECKSUM, which can increase the time it takes to restore the database.
- By enabling CHECKSUM, you can speed up the restore operations.

Migrate using Managed Instance link

- Azure SQL Managed Instance link feature offers a true online migration option compared to all other services and tools available.
- In addition, by partially running workloads on Azure, organizations can adopt a phased approach to cloud migration while still taking advantage of the benefits that Azure provides.
- While the link is designed to replicate one database per link, it can be configured to replicate multiple databases from a single SQL Server instance to one or more SQL managed instances, or even replicate the same database to multiple SQL managed instances.
- This is achieved by setting up multiple links, each corresponding to a database-managed instance pair.
- Hybrid flexibility with Azure SQL Managed Instance link feature
- Azure SQL Managed Instance link feature allows you to replicate your SQL Server databases hosted anywhere to Azure, and fail over to the cloud in the event of a disaster or major business disruption.
- Azure SQL Managed Instance link also ensures seamless failover between the primary and secondary databases.
- One of the advantages of using Azure SQL Managed Instance is that it's a platform as a service (PaaS), which means that the latest hardware maintenance, patching, and updates are applied and managed automatically by Azure.
- This ensures that your database environment is always up-to-date and secure, while also reducing the risk of downtime due to hardware failures or software vulnerabilities.



- As we can see above, the link feature uses distributed availability group (DAG) and it's scoped per database (one link per one database).
- This allows you to consolidate multiple parallel SQL Server databases into an Azure SQL Managed Instance or scale them out across multiple instances and regions worldwide.
- The link feature provides two types of replication:
- One-way replication.
- One-way replication is available for SQL Server versions 2016 and 2019 and allows you to replicate data one way from a SQL Server instance to your managed instance.
- Two-way replication.
- SQL Server 2022 provides a two-way replication feature, where you can replicate data between your managed instance and SQL Server instances, manually fail over during a disaster, and manually fail back after the disaster is mitigated.
- With SQL Server 2022, it supports online failover to Azure SQL Managed Instance and online failback to SQL Server by using the link directly.
- You can also fail back offline by backing up the database on the managed instance and restoring it to SQL Server.

Extended capabilities to the cloud

- In addition to migrate workloads, there are several ways to use the link feature and use Azure services and resources, which include:

Feature	Description
Offload read-only workloads	You may want to configure secondary replicas on your SQL Server to Azure to offload reporting needs. The link feature is database scoped, allowing for consolidation of read-only workloads in Azure, which can be used to bring data closer to customers in any supported region worldwide with minimum effort.
Automated backups	Secondary replicas running on Azure SQL Managed Instance are automatically backed up to your Azure Blob Storage account, which significantly reduces administrative efforts and improve reliability.
Business continuity	As a disaster recovery solution, the link feature allows you to fail over to Azure SQL Managed Instance and fail back after the disaster is mitigated.

Enable the link feature

- To configure the link feature, you must follow the same steps regardless of whether you're migrating to Azure SQL Managed Instance, configuring disaster recovery on the cloud, offloading workloads to Azure, or aiming to reduce backup operations and management costs.

- You can use either a wizard in SQL Server Management Studio (SSMS) or scripts.
- The main advantage of using scripts is that they can be automated, which can improve your deployment process, saving time and effort.
- Replicate a database by using Azure SQL Managed Instance link wizard available in SSMS.
- Replicate a database by using T-SQL and PowerShell scripts.
- There are a few SQL Server features that aren't supported by Azure SQL Managed Instance link.
- For example, you can't enable the link feature if the functionality that's used on the primary SQL Server database isn't supported on Azure SQL Managed Instance, such as file tables and file streams.
- For the full list of supported features, see Limitations of Azure SQL Managed Instance link.
- As we've seen, Azure SQL Managed Instance link feature enables organizations to confidently extend their SQL Server environments to Azure while also benefiting from the scalability, performance, and security features that Azure SQL Managed Instance offers.

Choosing between migration methods

- The choice between Managed Instance link and LRS depends on your specific requirements:

Consideration	Managed Instance link	Log Replay Service
Minimum downtime required	Best choice - cutover in seconds	Longer cutover, especially for Business Critical tier
Need to read data during migration	Supported	Not available - database is in restoring state
SQL Server version	2016 and later	2012 and later
SQL Server edition	Enterprise, Standard, Developer	All editions
Network configuration	Requires VPN or private endpoint setup	Works with public endpoint by default
Migration duration	Unlimited	Maximum 30 days
Reverse migration needed	Supported (depends on SQL MI update policy)	Not supported

- For most business-critical workloads targeting either General Purpose or Business Critical SQL Managed Instance, the Managed Instance link provides the best migration experience with minimal downtime.
- LRS is well-suited for general purpose workloads where some planned downtime is acceptable, or when migrating from older SQL Server versions or editions not supported by the Managed Instance link.

Move data to SQL Managed Instance

- Many migrations involve a period when the on-premises and the cloud database must be kept synchronized.
- For example, there might be a time when clients make changes to both databases.
- You've migrated the sports retail products database into Azure SQL Managed Instance.
- The website is already using the cloud database.
- You're starting to reconfigure clients to use the new database.
- You've decided to transition users to the new system in batches.
- For each team, you'll take time to resolve any problems before migrating the next users.
- This approach allows for troubleshooting and problem resolution without disrupting all users at once.
- Next, you'll reconfigure the data analysis system to use the new database in Azure.
- During this time, you want to ensure that the cloud and on-premises databases are synchronized every hour.

- You'll explore various methods for implementing data synchronization.
- These methods can also be employed to selectively migrate data, should you require only a subset of the tables to be transferred.
- This flexibility allows for a more tailored approach to data migration.
- Connectivity options with on-premises servers
- Often, you want to keep data in on-premises databases synchronized with Azure SQL Managed Instance.
- You might want to stage the migration of client applications to the new database, for example, which means there's a period when clients connect to both databases.
- Before you choose a data synchronization method, it's important to ensure you have connectivity that's secure.
- There are three different connectivity options available to establish communication between computers on-premises and resources in Azure.
- Point-to-Site.
- A Point-to-Site (P2S) VPN gateway connection lets you create a secure connection to your virtual network from an individual client computer.
- Site-to-Site.
- A Site-to-Site VPN gateway is used to connect an entire on-premises site to the Azure network.
- ExpressRoute.
- Azure ExpressRoute enables you to create private connections between Azure datacenters and on-premises infrastructure, or infrastructure in a colocation environment.
- ExpressRoute connections don't go over the public internet, and offer more reliability, faster speeds, lower latencies, and higher security than typical internet connections.

Public endpoint

- Public endpoint for SQL Managed Instance helps you connect to the database from the internet without using a VPN, and is designed for data communication only.
- Public endpoint for data can simultaneously coexist with the private endpoint.
- For security reasons, the implementation allows for Separation of Duties (SoD) between a database administrator and a network administrator when enabling the public endpoint.
- To enable public endpoint for managed instance, two steps are required.
- For SoD, you'll need two separate roles, with the following database and network permissions, to complete these steps:
- A database administrator who has role-based access control permissions in scope Microsoft.Sql/managedInstances/* must run a PowerShell script to enable public endpoint for managed instance.
- A network administrator who has role-based access control permissions in scope Microsoft.Network/* must open the port 3342 used by the public endpoint on the network security group (NSG), and provide a UDR route to avoid asymmetric routing.

Choose a synchronization method

- You can use many methods to synchronize data from a SQL Database managed instance to an on-premises server and back.

Native backup and restore

- You can restore a database in Azure SQL Managed Instance from an Azure Blob Storage file using Shared Access Signature (SAS).
- This involves creating a credential with access to Azure Blob Storage, and then using the BACKUP DATABASE command with the COPY_ONLY option.
- If your database is larger than 200 GB, you can use a striped backup by providing several URL locations.

```
SQL
```

```
BACKUP DATABASE YourDatabase TO URL = 'https://youraccount.blob.core.windows.net/yourcontainer/yourdatabase.bak'
WITH COPY_ONLY
```

- To restore the database in SQL Managed Instance:

```
SQL
```

```
RESTORE DATABASE YourDatabase FROM URL =
'https://youraccount.blob.core.windows.net/yourcontainer/yourdatabase.bak'
```

BACPAC file using SqlPackage

- A BACPAC file is essentially a zipped version of both your database's metadata and data.
- While this deployment method is compatible with SQL Database, SQL Managed Instance doesn't support migration via BACPAC within the Azure portal.
- As an alternative, the SQLPackage utility should be used with the BACPAC file.

Bulk Copy Program (BCP)

- The BCP utility is a command-line tool that exports tables to files so you can import them.
- Use this approach to migrate from a single SQL Database to SQL Managed Instance and back.

Azure Data Factory (ADF)

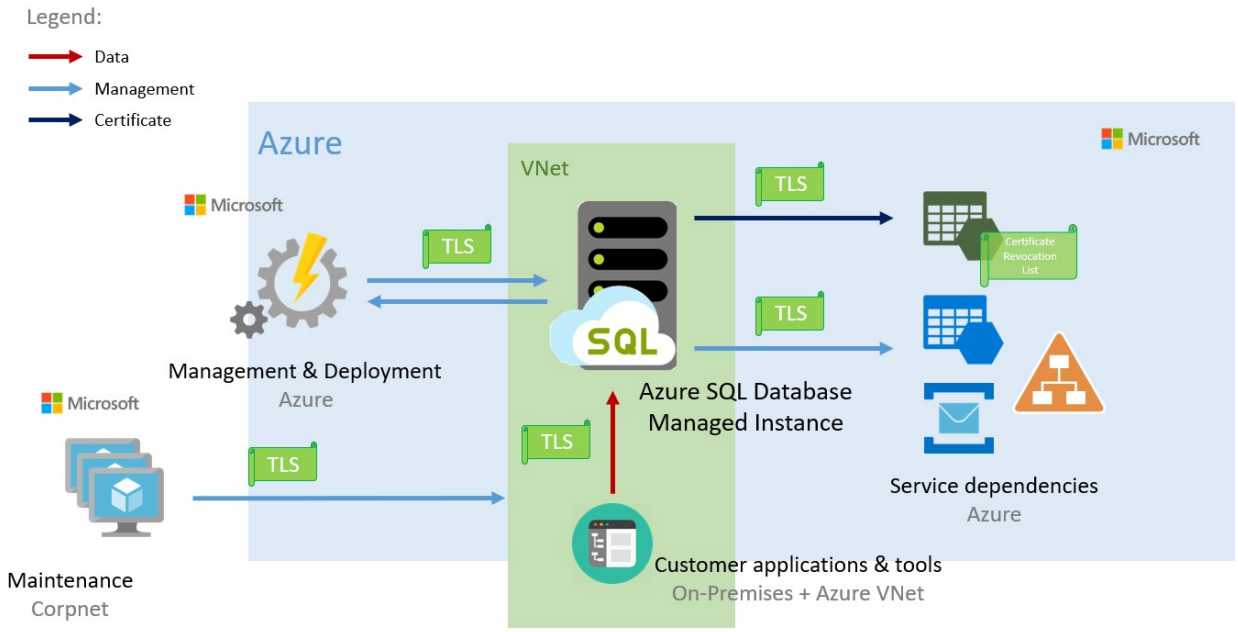
- Azure Data Factory is built for data movement and orchestration, with the focus on ingestion.
- ADF has the integration runtime support to run SSIS packages, and the public internet support for SQL Managed Instance.

Transactional replication

- Transactional replication is a way to move data between continuously connected database servers.
- The process begins with a snapshot of the publication database objects and data.
- Once the initial snapshot is taken, any subsequent changes to the data or schema at the Publisher are typically delivered to Azure SQL Managed Instance in near real-time as they occur.
- SQL Managed Instance is flexible because it can be a publisher, distributor, and subscriber.
- Replication is one of the few technologies that allows you to replicate parts of a table.
- We refer to these table parts as articles.
- This data is then sent to a distributor, which is a supplier of the data to any number of subscribers.

Requirements

- Connectivity uses SQL Authentication between replication participants.
- An Azure Storage Account share for the working directory used by replication.
- Open port 445 (TCP outbound) in the security rules of the managed instance subnet to access the Azure file share.
- Open port 1433 (TCP outbound) if the publisher or distributor is on a managed instance and the subscriber is on-premises.
- Connecting applications to a SQL managed instance
- A SQL managed instance must be placed inside an Azure virtual network subnet that's dedicated to managed instances.
- This deployment gives you a secure private IP address and the ability to connect to on-premises networks.



- Users and client applications can connect to the managed instance database through the Azure portal, PowerShell, Azure CLI, and the REST API.
- All communications are encrypted and signed using certificates.
- To check the trustworthiness of communicating parties, managed instances constantly verify these certificates through certificate revocation lists.
- If the certificates are revoked, the SQL managed instance closes the connections to protect the data.

Exercise - Migrate a SQL Server database to Azure SQL Managed Instance

- Now it's your chance to migrate a SQL Server database using Log Replay Service.
- In this exercise, you'll learn how to migrate a SQL Server database to Azure SQL Managed Instance using Log Replay Service.

Note

- To complete this exercise, you will need an Azure subscription.
- Launch the exercise and follow the instructions.

Summary

- In your sports clothing retail company, you needed an easy and quick way to migrate your products database into Azure.
- You chose Azure SQL Managed Instance because it provides a platform as a service (PaaS) implementation of SQL Server in the cloud.
- Azure SQL Database also has almost 100 percent compatibility with on-premises versions of SQL Server.
- You migrated the data, and ensured it was synchronized with the on-premises database so you could reconfigure client applications and the data analysis system.
- Before SQL Managed Instance was available, system architects who wanted to migrate a database to Azure had two options:
- You could use Azure SQL Database, which provided a full PaaS solution with the consequent reduction in administrative time.
- However, this often required significant modification of the database, because Azure SQL Database doesn't support all the features of SQL Server.
- You could install SQL Server on a virtual machine in Azure.

- However, this infrastructure as a service (IaaS) solution requires you to run, administer, and update the virtual machine, its operating system, and SQL Server yourself.
- You could expect to spend more time on administration.
- SQL Managed Instance provides a solution that's a full PaaS but supports almost all the features of SQL Server.
- This solution gives you the low administrative load you'd expect from Azure SQL Database but without the need to rewrite your database and change your client apps.
- For additional reading, you can refer to the following URLs:
- What is Azure SQL Managed Instance?
- Migrate SQL Server workloads to Azure SQL Database
- Tutorial: Migrate SQL Server to an Azure SQL Managed Instance online using DMS (classic)